

Core Language Working Group "ready" Issues for the November, 2025 meeting

References in this document reflect the section and paragraph numbering of document [WG21 N5014](#).

1670. `auto` as *conversion-type-id*

Section: 9.2.9.7 [[dcl.spec.auto](#)] **Status:** ready **Submitter:** Richard Smith **Date:** 2013-04-26 [N3690 comment FI 4](#)

(See also submission [#736](#).)

The current wording allows something like

```
struct S {  
    operator auto() { return 0; }  
};
```

If it is intended to be permitted, the details of its handling are not clear. Also, a similar syntax has been discussed as a possible future extension for dealing with proxy types in deduction which, if adopted, could cause confusion.

Additional note, November, 2013:

Doubt was expressed during the 2013-11-25 drafting review teleconference as to the usefulness of this provision. It is therefore being left open for further consideration after C++14 is finalized.

Notes from the February, 2014 meeting:

CWG continued to express doubt as to the usefulness of this construct but felt that if it is permitted, the rules need clarification.

Additional note (December, 2021):

See duplicate issue 2493 for additional details.

Additional note (August, 2025):

CWG appears to be leaning towards making declarations of `operator auto` ill-formed. Forwarding to EWG for approval of that direction via [paper issue #2409](#), by decision of the CWG chair.

Proposed resolution (approved by CWG 2025-11-07):

Change in 9.2.9.7.1 [[dcl.spec.auto.general](#)] paragraph 4 as follows:

A placeholder type can appear in the *decl-specifier-seq* or *type-specifier-seq* in the declared return type of a function declarator that declares a function **other than a conversion function (11.4.8.3 [[class.conv.fct](#)])**; the return type of the function is deduced from non-discarded return statements, if any, in the body of the function (8.5.2 [[stmt.if](#)]).

Additional notes (November, 2025)

See issue 1878 for conversion function templates with a placeholder return type.

2555. Ineffective redeclaration prevention for *using-declarators*

Section: 9.10 [[namespace.udcl](#)] **Status:** ready **Submitter:** Christof Meerwald **Date:** 2022-03-23

Consider:

```
template<int I>
struct C { };

struct B
{
    C<1> foo();
    C<1> bar();
};

struct D : B
{
    using B::foo;
    C<2> foo(this B &);

    using B::bar;
    C<2> bar(this D &);
};

struct DD : D
{
    using D::foo;
    using D::bar;
};

void bar(D d, DD dd)
{
    d.foo();
    dd.foo();

    d.bar();
    dd.bar();
}
```

Which functions are called?

Subclause 9.10 [[namespace.udecl](#)] paragraph 11 specifies:

The set of declarations named by a *using-declarator* that inhabits a class C does not include member functions and member function templates of a base class that correspond to (and thus would conflict with) a declaration of a function or function template in C.

The definition of "corresponds" considers the type of the implicit object parameter, which is a deviation from the status quo ante for a simple example like this one:

```
struct B {
    void f();    // #1
};
struct D : B {
    void f();
    using B::f; // should not name #1
};
```

Proposed resolution (approved by CWG 2025-10-10):

Change in 9.10 [[namespace.udecl](#)] paragraph 11 as follows:

The set of declarations named by a *using-declarator* that inhabits a class C does not include member functions and member function templates of a base class that, **when considered as members of C**, correspond to ~~(and thus would conflict with)~~ a declaration of a function or function template in C.

[*Example:*

```
struct B {
    virtual void f(int);
    virtual void f(char);
    void g(int);
    void h(int);
    void i();
    void j();
};

struct D : B {
    using B::f;
    void f(int);    // OK, D::f(int) overrides B::f(int)
```

```

using B::g;
void g(char); // OK

using B::h;
void h(int); // OK, D::h(int) hides B::h(int)

using B::i;
void i(this B &); // OK

using B::j;
void j(this D &); // OK, D::j() hides B::j()
};

void k(D* p)
{
    p->f(1); // calls D::f(int)
    p->f('a'); // calls B::f(char)
    p->g(1); // calls B::g(int)
    p->g('a'); // calls D::g(char)
    p->i(); // calls B::i, because B::i as a member of D is a better match than D::i
    p->j(); // calls D::j
}
...

```

2677. Replacing union subobjects

Section: 6.8.4 [\[basic.life\]](#) **Status:** ready **Submitter:** Richard Smith **Date:** 2022-12-06

The resolution to NB comment US 041 (C++20 CD) does not seem to have fully addressed the original issue, allowing:

```

union U { int i, j; };
U u;
new (&u) U{.i = 5};
int k = u.j; // OK! New U::i transparently replaces existing u.j!

```

The suggestion is to allow a newly-created complete object to transparently replace any object of the same type in the same storage, except for a potentially-overlapping subobject or a const complete object, and to allow a newly-created subobject to transparently replace only a corresponding subobject of an existing object.

Suggested resolution [SUPERSEDED]:

Change in 6.8.4 [\[basic.life\]](#) paragraph 8 as follows:

... An object o1 is transparently replaceable by an object o2 if:

- **o2 is a complete object and:**
 - the storage that o2 occupies exactly overlays the storage that o1 occupied, and
 - o1 and o2 are of the same type (ignoring the top-level cv-qualifiers), and
 - o1 is not a const, complete object, and
 - ~~neither o1 nor o2 is~~ **not** a potentially-overlapping subobject (6.8.2 [\[intro.object\]](#)), ~~and or~~
- ~~either o1 and o2 are both complete objects, or~~ o1 and o2 are **corresponding** direct subobjects of objects p1 and p2, respectively, and p1 is transparently replaceable by p2.

Additional notes (February, 2023)

The suggested resolution above does not handle the additional example in issue 2676.

Proposed resolution (approved by CWG 2025-11-07):

1. Change in 6.8.2 [\[intro.object\]](#) paragraph 2 as follows:

Objects can contain other objects, called subobjects. A subobject can be a member subobject (11.4 [\[class.mem\]](#)), a base class subobject (11.7 [\[class.derived\]](#)), or an array element. An object that is not a subobject of any other object is called a *complete object*. If an object is created in storage associated with a ~~member subobject or array element~~ **subobject** e (which may or may not be within its lifetime), the created object is a subobject of e's containing object if:

- the lifetime of e's containing object has begun and not ended, and
- the storage for the new object exactly overlays the storage location associated with e, and

- **e is not a potentially-overlapping subobject, and**
- the new object is of the same type as e (ignoring cv-qualification).

In this case, e and the created object are *corresponding direct subobjects*.

2. Change in 6.8.4 [basic.life] paragraph 8 as follows:

- ... An object o1 is transparently replaceable by an object o2 if **either**
- **o1 and o2 are complete objects for which:**
 - **o1 is not const,**
 - the storage that o2 occupies exactly overlays the storage that o1 occupied, and
 - o1 and o2 are of the same type (ignoring the top-level cv-qualifiers), **and or**
 - ~~o1 is not a const, complete object, and~~
 - ~~neither o1 nor o2 is a potentially-overlapping subobject (6.8.2 [intro.object]), and~~
 - ~~either o1 and o2 are both complete objects, or o1 and o2 are~~ **corresponding** ~~direct subobjects of objects p1 and p2, respectively, and p1 is transparently replaceable by p2 (6.8.2 [intro.object]) for which:~~
 - **the complete object of o1 is not const or**
 - **o1 is a mutable member subobject or a subobject thereof.**

2875. Missing support for round-tripping null pointer values through indirection/address operators

Section: C.7.4 [diff.expr] **Status:** ready **Submitter:** Richard Smith **Date:** 2024-03-21 [N5028 comment US 267-401](#)

C supports the following, C++ does not (see issues 232 and 2823):

```
void f() {
    char *p = nullptr;
    char *p2 = &*p;      // OK in C, undefined behavior in C++
    int a[5];
    int *q = &a[5];      // OK in C, undefined behavior in C++
}
```

This incompatibility should be documented in Annex C.

Proposed resolution (approved by CWG 2024-06-26):

Add a new paragraph to C.7.4 [diff.expr] as follows:

Affected subclause: 7.6.2.2 [expr.unary.op]

Change: In certain contexts, taking the address of a dereferenced null or past-the-end pointer value is well-defined in C (and yields the original pointer value), but results in undefined behavior in C++. For example:

```
void f() {
    char *p = nullptr;
    char *p2 = &*p;      // well-defined in C, undefined behavior in C++
    char *p3 = &p[0];    // well-defined in C, undefined behavior in C++
    int a[5];
    int *q = &a[5];      // well-defined in C, undefined behavior in C++
}
```

Rationale: Consistent treatment of lvalues in C++.

Effect on original feature: Well-formed and well-defined C code exhibits undefined behavior in C++.

Difficulty of converting: Syntactic transformation to pointer arithmetic and possible addition of a check for null pointer values.

How widely used: Occasionally.

CWG 2024-06-26

Implementations are required to diagnose undefined behavior in constant expressions. The issue is kept in review status to allow time for submitting a paper to EWG to make the &a[5] case well-defined. See also C23 6.5.3.2p3.

CWG 2025-10-24

No such paper has arrived in time for C++26. There is no prejudice against a paper in this area for C++29 or later.

2900. Deduction of non-type template arguments with placeholder types

Section: 13.10.3.6 [temp.deduct.type] Status: ready Submitter: Hubert Tong Date: 2024-06-05

(From submission #546.)

The deduction rule for non-type template parameters in 13.10.3.6 [temp.deduct.type] paragraph 20 seems lacking:

If P has a form that contains <i>, and if the type of i differs from the type of the corresponding template parameter of the template named by the enclosing *simple-template-id*, deduction fails. If P has a form that contains [i], and if the type of i is not an integral type, deduction fails. [Footnote: ...] ...

This wording does not address the situation when the declared type of i is a placeholder type, or when the type of the corresponding template parameter of the template named by the enclosing *simple-template-id* is a placeholder type.

Proposed resolution (approved by CWG 2025-11-06):

1. Change in 9.2.9.7.2 [dcl.type.auto.deduct] paragraph 3 as follows:

If the *placeholder-type-specifier* is of the form *type-constraint* opt auto, the deduced type T' replacing T is determined using the rules for template argument deduction. If the initialization is copy-list-initialization, a declaration of std::initializer_list shall precede (6.5.1 [basic.lookup.general]) the *placeholder-type-specifier*. Obtain P from T by replacing the occurrence of *type-constraint* opt auto either with a new invented type template parameter U or, if the initialization is copy-list-initialization, with std::initializer_list<U>. **If E is a value synthesized for a constant template parameter of type decl type(auto) (13.7.7.3 [temp.func.order]), the declaration is ill-formed. Otherwise, deduce** Deduce a value for U using the rules of template argument deduction from a function call (13.10.3.2 [temp.deduct.call]), where P is a function template parameter type and the corresponding argument is E. If the deduction fails, the declaration is ill-formed. Otherwise, T' is obtained by substituting the deduced U into P.

2. Change in 13.7.7.3 [temp.func.order] paragraph 3 as follows:

To produce the transformed template, for each type, constant, type template, variable template, or concept template parameter (including template parameter packs (13.7.4 [temp.variadic]) thereof) synthesize a unique type, value, class template, variable template, or concept, respectively, and substitute it for each occurrence of that parameter in the function type of the template.

~~[Note 1:~~ The type replacing ~~the~~ a placeholder in the type of the value synthesized for a constant template parameter is ~~also~~ a unique synthesized type. ~~—end note]~~

3. Change in 13.10.3.6 [temp.deduct.type] paragraph 20 as follows:

If P has a form that contains <i>, ~~and if the type of i differs from the type~~ **deduction fails unless the type of i is the same as that** of the corresponding template parameter **p in the specialization (from A)** of the template named by the enclosing *simple-template-id*, ~~deduction fails~~; **if the declared type of i contains a placeholder type, the corresponding template argument for the purposes of placeholder type deduction (9.2.9.7.2 [dcl.type.auto.deduct]) is an id-expression for p**. If P has a form that contains [i], and if the type of i is not an integral type, deduction fails. [Footnote: ...] ... [Example 13:

```
template<int i> class A { /* ... */ };
template<short s> void f(A<s>);
void k1() {
    A<1> a;
    f(a);      // error: deduction fails for conversion from int to short
    f<1>(a);    // OK
}
template<const short cs> class B { };
template<short s> void g(B<s>);
void k2() {
    B<1> b;
    g(b);      // OK, cv-qualifiers are ignored on template parameter types
}
```

```
template<auto> struct C;
template<long long x> void f(C<x> *);
void g(C<0LL> *ap) {
    f(ap);      // OK, deduces long long value from 0LL
}

template<int> struct D;
template<auto x> void f(D<x> *);
void g(D<0LL> *ap) {
```

```

    f(ap);    // OK, deduces x as an int value
}

template<int &> struct E;
template<auto x> void f(E<x> *);
int v;
void g(E<v> *bp) {
    f(bp);    // error: type int of x does not match the int & type of the template parameter in the E<v> specialization of E
}

template<const int &> struct F;
template<decltype(auto) x> void f(F<x> *);
int i;
void g(F<i> *ap) {
    f(ap);    // OK, deduces x as a constant template parameter of type const int &
}

template <decltype(auto) q> struct G;
template <auto x> long *f(G<x> *);    // #1
template <decltype(auto) x> short *f(G<x> *);    // #2

const int j = 0;
short *g(G<j> *ap) {    // OK, q has type const int &
    return f(ap);    // OK, only #2 matches
}

long *g(G<j> *ap) {    // OK, q has type int
    return f(ap);    // OK, #1 is more specialized
}

```

-- end example]

2917. Disallow multiple *friend-type-specifiers* for a friend template

Section: 13.1 [[temp.pre](#)] **Status:** ready **Submitter:** Ambrose T **Date:** 2024-07-30

(From submissions [#586](#) and [#593](#).)

Consider:

```

struct C {
    template <typename>
    friend class Foo, int;
};

```

This is obviously nonsense, but is not prohibited by the wording.

Also consider:

```

struct S {
    template <typename T>
    friend class Foo<T>::Nested;    // OK; see 13.7.5 [temp.friend] paragraph 5

    template <typename ...Ts>
    friend class Foo<Ts>::Nested...;    // makes no sense
};

```

Suggested resolution (2024-07-30). [SUPERSEDED]:

Change in 13.1 [[temp.pre](#)] paragraph 5 as follows:

In a *template-declaration*, explicit specialization, or explicit instantiation, the *init-declarator-list* in the declaration shall contain at most one declarator. When such a declaration is used to declare a class template, no declarator is permitted. **In a *template-declaration* whose declaration is a *friend-type-declaration*, the *friend-type-specifier-list* shall consist of exactly one *friend-type-specifier*; if it is a pack expansion (13.7.4 [[temp.variadic](#)]), any packs expanded by that pack expansion shall not have been introduced by the *template-declaration*.** [Example:

```

template<class ...>
struct C {
    struct Nested { };
};

template<class ... Us>
struct S {
    template <typename ...Ts>
    friend class C<Ts>::Nested...;    // error
    friend class C<Us>::Nested...;    // OK
};

```

-- end example]

The above resolution allows the following example:

```

template<class ... Us>
struct S {
    template<class T>
    friend class C<T, Us>::Nested...;
};

```

CWG was not convinced the above example was intended to be supported by paper P2893R3, which introduced pack expansions for friend declarations.

Proposed resolution (2024-08-16) [SUPERSEDED]:

(This is not a DR.)

(Issue 2862 modifies the same paragraph.)

Change in 13.1 [[temp.pre](#)] paragraph 5 as follows:

In a *template-declaration*, explicit specialization, or explicit instantiation, the *init-declarator-list* in the declaration shall contain at most one declarator. When such a declaration is used to declare a class template, no declarator is permitted. **In a *template-declaration* whose declaration is a *friend-type-declaration*, the *friend-type-specifier-list* shall consist of exactly one *friend-type-specifier* that is not a pack expansion.** [Example:

```

template<class ...>
struct C {
    struct Nested { };
};

template<class ... Us>
struct S {
    template <typename ...Ts>
    friend class C<Ts>::Nested...;    // error
    friend class C<Us>::Nested...;    // OK
};

```

-- end example]

CWG 2024-08-16

The proposed resolution above disallows a few examples from paper [P2893R3](#) (Variadic friends), such as

```

template<class... Ts>
struct VS {
    template<class U>
    friend class C<Ts>::Nested...; // now ill-formed
};

```

The adopted wording for P2893R3 makes the *friend-type-specifier* (not the entire *template-declaration*) the pattern that is expanded by the pack expansion, leading to the expansion

```

struct VS<T1, T2, T3> {
    template<class U>
    friend class C<T1>::Nested, class C<T2>::Nested, class C<T3>::Nested;
};

```

However, that violates the principle that a *template-declaration* declares exactly one entity (see issue 2862).

As an aside, the paper as adopted misrepresents the status of members of dependent types, which are covered by 13.7.5 [[temp.friend](#)] paragraph 5.

CWG welcomes a paper making the template friend cases valid, but such a facility would appear to require substantial changes to the normative wording, which a core issue is not equipped for.

CWG asks EWG to consent to the reduction in scope for the variadic friends facility, via paper issue [cplusplus/papers#2032](#).

CWG 2025-06-16

EWG agrees with the reduction in scope for the variadic friends facility, as proposed above.

Proposed resolution (approved by CWG 2025-11-04):

(This is not a DR.)

(Issue 2862 modifies the same paragraph.)

Change in 13.1 [[temp.pre](#)] paragraph 5 as follows:

In a *template-declaration*, explicit specialization, or explicit instantiation, the *init-declarator-list* in the declaration shall contain at most one declarator. When such a declaration is used to declare a class template, no declarator is permitted. **In a *template-declaration* whose declaration is a *friend-type-declaration*, the *friend-type-declaration* shall be of the form**

```
friend friend-type-specifier ;
```

[Example:

```
template<class ...>
struct C {
    struct Nested { };
};

template<class ... Us>
struct S {
    template <typename ...Ts>
    friend class C<Ts>::Nested...;    // error
    friend class C<Us>::Nested...;    // OK, not a template-declaration
};
```

-- end example]

2923. Note about infinite loops and execution steps

Section: 6.10.2.3 [[intro.progress](#)] **Status:** ready **Submitter:** Simon Cooksey **Date:** 2024-08-12 [N5028 comment US 23-043](#)

Given that certain infinite loops are no longer undefined behavior (see P2809R3 (Trivial infinite loops are not Undefined Behavior), adopted in March 2024), the note in 6.10.2.3 [[intro.progress](#)] paragraph 5 claiming there are no executions without eventual execution steps should be struck.

Proposed resolution (approved by CWG 2024-09-13 and 2025-10-10):

Remove 6.10.2.3 [[intro.progress](#)] paragraph 5:

~~[Note 4: Because of this and the preceding requirement regarding what threads of execution have to perform eventually, it follows that no thread of execution can execute forever without an execution step occurring. —end note]~~

CWG 2024-11-19

The definition of "execution step" in 6.10.2.3 [[intro.progress](#)] paragraph 3 does not include the invocation of `std::this_thread::yield`. However, trivial infinite loops do nothing but yield, and are not undefined behavior. Therefore, the note is believed to be factually wrong. CWG seeks advice from SG1 for resolving the inconsistency, via [paper issue #2139](#).

2929. Lifetime of trivially-destructible static or thread-local objects

Section: 6.10.3.4 [[basic.start.term](#)] **Status:** ready **Submitter:** Mathias Stearn **Date:** 2024-08-05

Issue 2256 (adopted in February, 2019) changed the rules such that the lifetimes of objects do not depend on whether the destructor is trivial or not. However, the motivation of that issue mentioned only objects with automatic storage duration. To be able to avoid order-of-destruction issues across translation units, it would be useful to never end the lifetime of trivially-destructible objects with static storage duration and to delay the end of the lifetime of such objects with thread storage duration as long as possible.

Possible resolution [SUPERSEDED]:

1. Change in 6.10.3.4 [basic.start.term] paragraph 1 as follows:

Constructed **class** objects (9.5 [dcl.init]) with static storage duration **and having a non-trivial destructor (11.4.7 [class.dtor])** are destroyed and functions registered with std::atexit are called as part of a call to std::exit (17.5 [support.start.term]). The call to std::exit is sequenced before the destructions and the registered functions.

2. Change in 6.10.3.4 [basic.start.term] paragraph 2 as follows:

Constructed **class** objects with thread storage duration within a given thread **and having a non-trivial destructor (11.4.7 [class.dtor])** are destroyed as a result of returning from the initial function of that thread and as a result of that thread calling std::exit. The destruction of ~~all~~ **those** constructed objects with thread storage duration within that thread strongly happens before **releasing the storage for objects with thread storage duration within that thread, which in turn strongly happens before** destroying any object with static storage duration.

CWG 2024-09-27

Also allow for constant destruction, consider arrays of class type, and adjust the happens-before requirements.

Proposed resolution (approved by CWG 2024-10-11) [SUPERSEDED]:

1. Change in 6.10.3.4 [basic.start.term] paragraph 1 as follows:

Constructed **complete** objects (9.5 [dcl.init]) with static storage duration **and not having constant destruction (7.7 [expr.const])** are destroyed and functions registered with std::atexit are called as part of a call to std::exit (17.5 [support.start.term]). The call to std::exit is sequenced before the destructions and the registered functions.

2. Change in 6.10.3.4 [basic.start.term] paragraph 2 as follows:

Constructed **complete** objects with thread storage duration within a given thread **and not having constant destruction (7.7 [expr.const])** are destroyed as a result of returning from the initial function of that thread and as a result of that thread calling std::exit. **The destruction of those constructed objects within that thread is sequenced before releasing the storage for all objects with thread storage duration within that thread (6.8.6.3 [basic.stc.thread]). Also, the** ~~The~~ destruction of ~~all~~ **those** constructed objects ~~with thread storage duration~~ within that thread strongly happens before destroying any object with static storage duration.

CWG 2024-11-22

Continue to actually destroy constant-destruction objects, but do so after all dynamic destruction is done.

Proposed resolution (approved by CWG 2025-11-04):

1. Change in 6.10.3.4 [basic.start.term] paragraph 1 as follows:

Constructed **complete** objects (9.5 [dcl.init]) with static storage duration are destroyed and functions registered with std::atexit are called as part of a call to std::exit (17.5 [support.start.term]). The call to std::exit is sequenced before the destructions and the registered functions.

2. Change in 6.10.3.4 [basic.start.term] paragraph 2 as follows:

Constructed **complete** objects with thread storage duration within a given thread are destroyed as a result of returning from the initial function of that thread and as a result of that thread calling std::exit. **The destruction of those constructed objects is sequenced before releasing the storage for any object with thread storage duration within that thread (6.8.6.3 [basic.stc.thread]).** ~~The~~ The destruction of ~~all~~ **those** constructed objects ~~with thread storage duration within that thread~~ strongly happens before destroying any object with static storage duration.

3. Add a paragraph after 6.10.3.4 [basic.start.term] paragraph 2:

The destruction of a complete object with thread storage duration within a given thread and having constant destruction (7.7 [expr.const]) is sequenced after the destruction of any other complete object with thread storage duration within the thread. The destruction of a complete object with static storage duration and having constant destruction is sequenced after the destruction of any other complete object with static storage duration and after any call to a function passed to

`std::atexit`. The sequencing rules in the remainder of this subclause apply only to complete objects not having constant destruction.

4. Do not change in 6.10.3.4 [\[basic.start.term\]](#) paragraph 3:

If the completion of the constructor or dynamic initialization of an object with static storage duration strongly happens before that of another, the completion of the destructor of the second is sequenced before the initiation of the destructor of the first. If the completion of the constructor or dynamic initialization of an object with thread storage duration is sequenced before that of another, the completion of the destructor of the second is sequenced before the initiation of the destructor of the first. If an object is initialized statically, the object is destroyed in the same order as if the object was dynamically initialized. For an object of array or class type, all subobjects of that object are destroyed before any block variable with static storage duration initialized during the construction of the subobjects is destroyed. If the destruction of an object with static or thread storage duration exits via an exception, the function `std::terminate` is called (14.6.2 [\[except.terminate\]](#)).

2941. Lifetime extension for function-style cast to reference type

Section: 6.8.7 [\[class.temporary\]](#) **Status:** ready **Submitter:** Hubert Tong **Date:** 2024-10-18 [N5028 comment CA 041](#)

(From submission [#625](#).)

Issue 2894 will clarify that a function-style cast to reference type produces a glvalue, not a prvalue. However, 6.8.7 [\[class.temporary\]](#) does not specify lifetime-extension for this case, even though implementations uniformly do extend the lifetime.

For example:

```
int glob;
struct A {
    constexpr ~A() { p = &glob; }
    int *p;
};
constexpr int f() {
    typedef const A &AR;
    const A &ref = AR{0};
    delete ref.p;
    return 0;
}
extern constexpr int x = f(); // okay
```

Proposed resolution (approved by CWG 2025-11-03):

Change in 6.8.7 [\[class.temporary\]](#) paragraph 6 as follows:

- a
 - `const_cast` (7.6.1.11 [\[expr.const.cast\]](#)),
 - ...
- an explicit type conversion (functional notation) (7.6.1.4 [\[expr.type.conv\]](#)) to a reference type whose initializer is a *braced-init-list* where the reference is
 - bound directly to the glvalue result of one of these expressions (necessarily the sole element of the *braced-init-list*) or
 - bound to the result of a temporary materialization conversion,
- a conditional expression (7.6.16 [\[expr.cond\]](#)) that is a glvalue where the second or third operand is one of these expressions, or

3001. Inconsistent restrictions for `static_cast` on pointers to out-of-lifetime objects

Section: 6.8.4 [\[basic.life\]](#) **Status:** ready **Submitter:** Cody Miller **Date:** 2025-02-26

According to 6.8.4 [\[basic.life\]](#) paragraph 7, converting a pointer-to-derived to pointer-to-base for an out-of-lifetime object is allowed implicitly, but disallowed using `static_cast`. That seems inconsistent.

Furthermore, base classes of virtual base classes should get the same treatment as virtual base classes themselves.

Proposed resolution (approved by CWG 2025-09-26):

1. Change 7.3.12 [\[conv.ptr\]](#) paragraph 3 as follows:

... Otherwise, if B is a virtual base class of D **or is a base class of a virtual base class of D** and v does not point to an object whose type is similar (7.3.6 [\[conv.qual\]](#)) to D and that is within its lifetime or within its period of construction or destruction (11.9.5 [\[class.ctor\]](#)), the behavior is undefined. ...

2. Change in 6.8.4 [\[basic.life\]](#) paragraph 7 as follows:

Before the lifetime of an object has started but after the storage which the object will occupy has been allocated [
Footnote: ...] ... The program has undefined behavior if

- the pointer is used as the operand of a *delete-expression*,
- the pointer is used to access a non-static data member or call a non-static member function of the object, **or**
- the pointer is **implicitly** converted (7.3.12 [\[conv.ptr\]](#), 7.6.1.9 [\[expr.static.cast\]](#)) to a pointer to a virtual base class **or to a base class thereof**, or
- ~~the pointer is used as the operand of a *static_cast* (7.6.1.9 [\[expr.static.cast\]](#)), except when the conversion is to pointer to cv void, or to pointer to cv void and subsequently to pointer to cv char, cv unsigned char, or cv std::byte (17.2.1 [\[cstdint.syn\]](#)), or~~
- the pointer is used as the operand of a *dynamic_cast* (7.6.1.7 [\[expr.dynamic.cast\]](#)).

3002. Template parameter/argument confusion

Section: 13.8.3.7 [\[temp.dep.temp\]](#) **Status:** ready **Submitter:** Hubert Tong **Date:** 2025-02-02

(From submission [#668](#).)

Subclause 13.8.3.7 [\[temp.dep.temp\]](#) paragraph 4 confuses template parameters and arguments.

Proposed resolution (approved by CWG 2025-09-12):

Change in 13.8.3.7 [\[temp.dep.temp\]](#) paragraph 4 as follows:

A template template **parameter** **argument** is dependent if it names a template parameter or its terminal name is dependent.

3004. Pointer arithmetic on array of unknown bound

Section: 7.7 [\[expr.const\]](#) **Status:** ready **Submitter:** A. Jiang **Date:** 2025-03-07

(From submission [#680](#).)

Consider:

```
extern const int arr[];
constexpr const int *p = arr + N; // #1
constexpr int arr[2] = {0, 1};    // #2
constexpr int k = *p;             // #3
```

What is the outcome here? Accept for $N == 0$ and otherwise #1 is non-constant (clang)? Always accept #1, but reject #3 for out-of-bounds accesses (gcc)? Always accept #1, but always reject #3, even for in-bound accesses (EDG)? Reject #2 (MSVC)?

Proposed resolution (approved by CWG 2025-09-12):

CWG chose "Accept for $N == 0$ and otherwise #1 is non-constant".

Add a bullet after 7.7 [\[expr.const\]](#) bullet 10.15 as follows:

- ...
- a *reinterpret_cast* (7.6.1.10 [\[expr.reinterpret.cast\]](#));
- pointer arithmetic (7.6.6 [\[expr.add\]](#)) where one (possibly converted) operand points to the first element of an array of unknown bound and the other (possibly converted) operand is of integral type with non-zero value;
- ...

3005. Function parameters should never be name-independent

Section: 6.4.1 [[basic.scope.scope](#)] **Status:** ready **Submitter:** Jan Schultke **Date:** 2025-03-10

(From submission [#681](#).)

According to [P2169R4](#) (A nice placeholder with no name):

In Varna, EWG decided not to support function parameter names and requires clauses param-eter as these can remained unnamed without semantics differences.

However, the normative wording does permit function parameters to be name-independent, contrary to the express design intent.

Proposed resolution (approved by CWG 2025-09-12):

Change in 6.4.1 [[basic.scope.scope](#)] bullet 5.1 as follows:

A declaration is name-independent if its name is `_` (u+005f low line) and it declares

- a variable, **other than a function parameter**, with automatic storage duration,
- a structured binding with no *storage-class-specifier* and not inhabiting a namespace scope,
- the variable introduced by an *init-capture*, or
- a non-static data member of other than an anonymous union.

3008. Missing Annex C entry for void object declarations

Section: C.7.6 [[diff.dcl](#)] **Status:** ready **Submitter:** Jiang An **Date:** 2025-03-13

(From submission [#682](#).)

C allows

```
extern void x;
```

C++ does not, as clarified by issue 2475 in 9.1 [[dcl.pre](#)] paragraph 8.

This deviation warrants an entry in Annex C.

Proposed resolution (approved by CWG 2025-09-12):

Add a new paragraph to C.7.6 [[diff.dcl](#)] as follows:

Affected subclause: 9.1 [[dcl.pre](#)]

Change: In C++, no declaration of a variable can have `cv void` type. In C, this is allowed, unless the declaration is a definition.
[Example:

```
extern void x;    // valid C, invalid in C++
```

-- end example]

Rationale: Stricter type checking in C++.

Effect on original feature: Deletion of semantically well-defined feature.

Difficulty of converting: Syntactic transformation.

How widely used: Seldom.

3011. Parenthesized aggregate initialization for *new-expressions*

Section: 7.6.2.8 [[expr.new](#)] **Status:** ready **Submitter:** Benjamin Sch. **Date:** 2025-03-17

(From submission [#685](#).)

Subclause 7.6.2.8 [\[expr.new\]](#) paragraph 8 and paragraph 9 contain special cases involving aggregate initialization using *braced-init-lists*, which need to be amended for parenthesized aggregate initialization. The corresponding rules for the initialization of variables are in 9.5.1 [\[dcl.init.general\]](#) bullet 16.5 for arrays and in 9.5.1 [\[dcl.init.general\]](#) bullet 16.6.2.2 for aggregate classes.

Possible resolution (March, 2025) [SUPERSEDED]:

1. Change in 7.6.2.8 [\[expr.new\]](#) bullet 8.4 as follows:

If the expression in a *nopttr-new-declarator* is present, it is implicitly converted to `std::size_t`. The value of the expression is invalid if

- ...
- the *new-initializer* is a *braced-init-list* **present** and the number of array elements for which initializers are provided (including the terminating `'\0'` in a *string-literal* (5.13.5 [\[lex.string\]](#))) **in a *braced-init-list* or parenthesized *expression-list*** exceeds the number of elements to initialize.

2. Change in 7.6.2.8 [\[expr.new\]](#) paragraph 9 as follows:

If the allocated type is an array, the *new-initializer* is a *braced-init-list* **present and is not `()`**, and the *expression* is potentially-evaluated and not a core constant expression, the semantic constraints of **copy**-initializing a hypothetical element of the array **from an empty initializer list** are checked **as follows**:

- **If the *new-initializer* is a *braced-init-list*, the hypothetical element is copy-initialized from an empty initializer list** (9.5.5 [\[dcl.init.list\]](#)).
- If the *new-initializer* is a parenthesized *expression-list*, the hypothetical element is value-initialized (9.5.1 [\[dcl.init.general\]](#)).

[Note 5: The array can contain more elements than there are elements in the *braced-init-list new-initializer*, requiring initialization of the remainder of the array elements **from an empty initializer list as appropriate**. — end note]

Possible resolution (per CWG reflector review starting 2025-05-07) [SUPERSEDED]:

1. Change in 7.6.2.8 [\[expr.new\]](#) bullet 8.4 as follows:

If the expression in a *nopttr-new-declarator* is present, it is implicitly converted to `std::size_t`. The value of the expression is invalid if

- ...
- the *new-initializer* is a *braced-init-list* **present** and the number of array elements for which initializers are provided (including the terminating `'\0'` in a *string-literal* (5.13.5 [\[lex.string\]](#))) **in a *braced-init-list* or parenthesized *expression-list*** exceeds the number of elements to initialize.

2. Change in 7.6.2.8 [\[expr.new\]](#) paragraph 9 as follows:

If the allocated type is an array, the *new-initializer* is a *braced-init-list* **present**, and the *expression* is potentially-evaluated and not a core constant expression, the semantic constraints of **copy**-initializing a hypothetical element of the array **from an empty initializer list** are checked **as follows**:

- **If the *new-initializer* is a *braced-init-list*, the hypothetical element is copy-initialized from an empty initializer list** (9.5.5 [\[dcl.init.list\]](#)).
- If the *new-initializer* is a parenthesized *expression-list*, the hypothetical element is value-initialized (9.5.1 [\[dcl.init.general\]](#)).

[Note 5: The array can contain more elements than there are elements in the *braced-init-list new-initializer*, requiring initialization of the remainder of the array elements **from an empty initializer list as appropriate**. — end note]

Proposed resolution (approved by CWG 2025-10-10):

1. Change in 7.6.2.8 [\[expr.new\]](#) bullet 8.4 as follows:

If the expression in a *nopttr-new-declarator* is present, it is implicitly converted to `std::size_t`. The value of the expression is invalid if

- ...
- the *new-initializer* is a *braced-init-list* **or a parenthesized *expression-list*** and the number of array elements for which initializers are provided (including the terminating `'\0'` in a *string-literal* (5.13.5 [\[lex.string\]](#))) exceeds the number of elements to initialize.

2. Change in 7.6.2.8 [\[expr.new\]](#) paragraph 9 as follows:

If the allocated type is an array, the *new-initializer* is a *braced-init-list* **or a parenthesized *expression-list***, and the *expression* is potentially-evaluated and not a core constant expression, the semantic constraints of **copy**-initializing a hypothetical element of the array **from an empty initializer list** are checked **as follows**:

- If the *new-initializer* is a *braced-init-list*, the hypothetical element is copy-initialized from an empty initializer list (9.5.5 [dcl.init.list]).
- Otherwise, the hypothetical element is value-initialized (9.5.1 [dcl.init.general]).

[Note 5: The array can contain more elements than there are elements in the *braced-init-list new-initializer*, requiring initialization of the remainder of the array elements from an empty initializer list as appropriate. — end note]

3032. Template argument disambiguation

Section: 13.4.1 [temp.arg.general] **Status:** ready **Submitter:** Hubert Tong **Date:** 2025-02-03 [N5028 comment CA 092](#)

(From submission [#669](#).)

For the *template-argument* grammar in 13.3 [temp.names], there is a parsing ambiguity introduced by P2841R7 between *nested-name-specifier*_{opt} *template-name* and *type-id* (via *simple-type-specifier*) naming a placeholder for deduced class type (9.2.9.8 [dcl.type.class.deduct]). For example:

```
void g();

template <typename T>
struct A {
    static void f() {
        g<T::TT>((A *)0);
    }
};

template <typename T> void g(void *);
template <auto> void g(void *);
template <template <typename> class> void g(void *);

struct Expr { enum { TT = 0 }; };
struct Type { using TT = int; };
struct Tmpl { template <typename> struct TT; };
void h() {
    A<Expr>::f(); // all accept
    A<Type>::f(); // EDG, MSVC accept; GCC, Clang rejects
    A<Tmpl>::f(); // EDG, MSVC accept; GCC, Clang rejects
}
```

P1787R6 established the direction that the template disambiguator is needed only when introducing a *template-argument-list* (13.3 [temp.names] paragraph 6). (Reflection made this assumption false.) All examples should be accepted.

See 13.8.2 [temp.local] for the use of an injected-class-name as a template argument.

Example:

```
void g();

template <typename T>
struct A {
    static void f() {
        g<A>((A *)0); // all accept
        g<A>((A *)0, 0); // Clang, GCC, EDG accept; MSVC rejects
    }
};

template <typename T> void g(void *);
template <template <typename> class> void g(void *, int);

void h() { A<int>::f(); }
```

Proposed resolution (approved by CWG 2025-11-06):

Strategy: Introduce a new grammar production akin *reflection-name*, as a preferred option for *template-argument*, and say what it means in the various cases.

1. Change in 13.3 [temp.names] paragraph 1 as follows:

```
template-argument:
    template-argument-name
    constant-expression
```

```

type-id
nested-name-specifieropt template-name
nested-name-specifier template template-name

template-argument-name:
  nested-name-specifieropt identifier
  nested-name-specifier template identifier

```

2. Insert a new paragraph before 13.3 [temp.names] paragraph 8 as follows:

The component names of a *template-argument-name* are those of its *nested-name-specifier* (if any) and its *identifier*.

A *template-id* is valid if ...

3. Insert a new paragraph before 13.4.1 [temp.arg.general] paragraph 3 as follows:

If a *template-argument* *A* matches the form *template-argument-name*, it is interpreted as such; the *identifier* is looked up and its meaning is determined as follows:

- If lookup finds an injected-class-name (13.8.2 [temp.local]), then:
 - When *A* is for a type template template parameter, *A* denotes the corresponding class template.
 - Otherwise, it denotes a *type-name*.
- Otherwise, if lookup finds a template, *A* denotes that template.
- Otherwise, if lookup finds a type alias or a type, *A* denotes the underlying type and is interpreted as a *type-id*.
- Otherwise, *A* is interpreted as an expression.

In a *template-argument*, an ambiguity between a *type-id* and an expression is resolved to a *type-id*, regardless of the form of the corresponding *template-parameter*...

3043. Lifetime extension for temporaries in expansion statements

Section: 6.8.7 [class.temporary] **Status:** ready **Submitter:** Jakub Jelinek **Date:** 2025-07-07 [N5028 comment US 21-038](#)

(From submission [#724](#).)

Subclause 6.8.7 [class.temporary] paragraph 7 and paragraph 8 specify:

The fourth context is when a temporary object is created in the *for-range-initializer* of either a range-based for statement or an enumerating expansion statement (8.7 [stmt.expand]). If such a temporary object would otherwise be destroyed at the end of the *for-range-initializer* full-expression, the object persists for the lifetime of the reference initialized by the *for-range-initializer*.

The fifth context is when a temporary object is created in the *expansion-initializer* of an iterating or destructuring expansion statement. If such a temporary object would otherwise be destroyed at the end of that *expansion-initializer*, the object persists for the lifetime of the reference initialized by the *expansion-initializer*, if any.

There are a number of problems with this phrasing:

- The original idea of lifetime extension in a *for-range-initializer* was to ensure that temporaries defining a range would survive across all iterations. For an enumerating expansion statement, there is no overarching range expression; each element of the *expression-list* is considered separately. Furthermore, there is no *for-range-initializer* in the corresponding grammar to start with.
- For an iterating expansion statement, the involved variables are *constexpr*, so no dangling pointers or similar could escape from the constant evaluation to start with. The necessity for temporary lifetime extension is thus unclear.

Proposed resolution (approved by CWG 2025-11-04):

Change in 6.8.7 [class.temporary] paragraph 7 and paragraph 8 as follows:

The fourth context is when a temporary object is created in the *for-range-initializer* of either a range-based for statement or an enumerating expansion statement (8.7 [stmt.expand]). If such a temporary object would otherwise be destroyed at the end of the *for-range-initializer* full-expression, the object persists for the lifetime of the reference initialized by the *for-range-initializer*.

The fifth context is when a temporary object is created in an element *E* of the *expansion-init-list* of an enumerating expansion statement (8.7 [stmt.expand]). If such a temporary object would otherwise be destroyed at the end of the full-expression of *E*, the object persists for the lifetime of the *for-range-declaration* initialized from *E*.

The **fifth** **sixth** context is when a temporary object is created in the *expansion-initializer* of **an iterating or a** destructuring expansion statement. If such a temporary object would otherwise be destroyed at the end of that *expansion-initializer*, the object persists for the lifetime of the reference initialized by the *expansion-initializer*, if any.

The **sixth** **seventh** context is when ...

3044. Iterating expansion statements woes

Section: 8.7 [[stmt.expand](#)] **Status:** ready **Submitter:** Jakub Jelinek **Date:** 2025-07-08

(From submissions [#725](#) and [#730](#).)

The rewrite of iterating expansion statements contains:

```
static constexpr auto iter = begin + i;
```

where the type of *i* is unspecified, but overload resolution of *+* depends on the type of *i*.

Furthermore, the rewrite contains

```
for (auto i = begin ; i != end ; ++i, ++result);
```

which might invoke an overloaded comma operator, which is undesirable.

Furthermore, because of the use of *static* in the rewrite, the example in 8.7 [[stmt.expand](#)] paragraph 7 is ill-formed.

Possible resolution (July 2025) [SUPERSEDED]:

1. Change in 8.7 [[stmt.expand](#)] bullet 5.2 as follows:

Otherwise, if *S* is an iterating expansion statement, *S* is equivalent to:

```
{
  init-statement
  static constexpr auto&& range = expansion-initializer ;
  static constexpr auto begin = begin-expr;    // see 8.6.5 [stmt.ranged]
  static constexpr auto end = end-expr;        // see 8.6.5 [stmt.ranged]
  S0
  .
  .
  SN-1
}
```

where *N* is the result of evaluating the expression

```
[] consteval {
  std::ptrdiff_t result = 0;
  for (auto i = begin ; i != end ; ++i, ++result) ++result;
  return result;    // distance from begin to end
}()
```

and *S_i* is

```
{
  static constexpr auto iter = begin + i i;
  for-range-declaration = *iter ;
  compound-statement
}
```

The variables *range* , *begin* , *end* , and *iter* are defined for exposition only. **The identifier *i* is considered to be a prvalue of type `decltype(begin - begin)`; the program is ill-formed if *i* is not representable as such a value.** [Note 1 : The instantiation is ill-formed if *range* is not a constant expression (7.7 [[expr.const](#)]). -- end note]

2. No change in 8.7 [[stmt.expand](#)] paragraph 7:

[Example 2:

```
constexpr int f() {
  constexpr std::array<int, 3> arr {1, 2, 3};
  int result = 0;
```

```

        template for (constexpr int s : arr) {           // OK, iterating expansion statement
            result += sizeof(char[s]);
        }
        return result;
    }
    static_assert(f() == 6);

    —end example]

```

Proposed resolution (approved by CWG 2025-11-06):

Change in 8.7 [stmt.expand] bullet 5.2 as follows:

Otherwise, if S is an iterating expansion statement, S is equivalent to:

```

{
    init-statement
    static constexpr auto&& range = expansion-initializer ;
    static constexpr auto begin = begin-expr;           // see 8.6.5 [stmt.ranged]
    static constexpr auto end = end-expr;               // see 8.6.5 [stmt.ranged]
    S0
    .
    .
    .
    SN-1
}

```

where N is the result of evaluating the expression

```

[] consteval {
    std::ptrdiff_t result = 0;
    for (auto i = begin ; i != end ; ++i, ++result) ++result;
    return result;    // distance from begin to end
}()

```

and Si is

```

{
    static constexpr auto iter = begin + i; decltype(begin - begin){i};
    for-range-declaration = *iter ;
    compound-statement
}

```

The variables range , begin , end , and iter are defined for exposition only. The identifier i is considered to be a prvalue of type std::ptrdiff_t; the program is ill-formed if i is not representable as such a value. [Note 1 : The instantiation is ill-formed if range is not a constant expression (7.7 [expr.const]). -- end note]

3045. Regularizing environment interactions of expansion statement

Section: 6.4.3 [basic.scope.block] **Status:** ready **Submitter:** Jakub Jelinek **Date:** 2025-07-18

(From submission #729.)

Consider:

```

template for (auto x : whatever) {
    int x = 42;
}

```

This is valid, but the parallel for-range-statement is not:

```

for (auto g : whatever) {
    int g = 42;
}

```

Furthermore, due to the possibly surprising rewriting nature of expansion statements, 9.13.5 [dcl.attr.fallthrough] should prohibit [[fallthrough]].

Proposed resolution (approved by CWG 2025-09-12):

1. Change in 6.4.3 [basic.scope.block] paragraph 2 as follows:

If a declaration that is not a name-independent declaration and that binds a name in the block scope S of a

- *compound-statement* of a *lambda-expression*, *function-body*, or *function-try-block*,
- substatement of a selection **or**, iteration, **or expansion** statement that is not itself a selection **or**, iteration, **or expansion** statement, or
- handler of a *function-try-block*

potentially conflicts with a declaration whose target scope is the parent scope of S, the program is ill-formed.

2. Change in 9.13.5 [dcl.attr.fallthrough] paragraph 1 as follows:

The *attribute-token* fallthrough may be applied to a null statement (8.3 [stmt.expr]); such a statement is a fallthrough statement. No *attribute-argument-clause* shall be present. A fallthrough statement may only appear within an enclosing switch statement (8.5.3 [stmt.switch]). The next statement that would be executed after a fallthrough statement shall be a labeled statement whose label is a case label or default label for the same switch statement and, if the fallthrough statement is contained in an iteration statement, the next statement shall be part of the same execution of the substatement of the innermost enclosing iteration statement. The program is ill-formed if there is no such statement. **The innermost enclosing switch statement of a fallthrough statement S shall be contained in the innermost enclosing expansion statement (8.7 [stmt.expand]) of S, if any.**

3048. Empty destructuring expansion statements

Section: 8.7 [stmt.expand] **Status:** ready **Submitter:** Jakub Jelinek **Date:** 2025-08-09

(From submission #743.)

Consider:

```
struct S { };
void f() {
    S s;
    template for (auto x : s) { }
```

This destructuring expansion statement is ill-formed, because the syntactic rewrite contains

```
auto&& [] = s ;
```

which is an ill-formed structured binding declaration.

Empty expansion statements are allowed for the other two kinds of expansion statements.

Proposed resolution (approved by CWG 2025-09-12):

Change in 8.7 [stmt.expand] bullet 5.3 as follows:

- Otherwise, S is a destructuring expansion statement and, **if N is 0, S is equivalent to:**

```
{
    init-statement
    constexpr auto&& range = expansion-initializer ;
}
```

otherwise, S is equivalent to: ...

3053. Allowing #undef likely

Section: 15.7.1 [cpp.replace.general] **Status:** ready **Submitter:** Jakub Jelinek **Date:** 2025-07-31 [N5028 comment CA 104](#)

(From submission #734.)

P2843 moved a prohibition to #define keywords to the core language; it is now appearing in 15.7.1 [cpp.replace.general] paragraph 9:

A translation unit shall not #define or #undef names lexically identical to keywords, to the identifiers listed in Table 4, or to the *attribute-tokens* described in 9.13 [dcl.attr], except that the names likely and unlikely may be defined as function-like macros.

This allows

```
#define likely(x) x
#define unlikely(a, b) a + b
```

but prohibits

```
#undef likely
#undef unlikely
```

even though guidelines for good macro hygiene suggest to `#undef` a macro past its region of use.

Proposed resolution (approved by CWG 2025-10-24):

Change in 15.7.1 [[cpp.replace.general](#)] paragraph 9 as follows:

A translation unit shall not `#define` or `#undef` names lexically identical to keywords, to the identifiers listed in Table 4, or to the *attribute-tokens* described in 9.13 [[decl.attr](#)], except that the names `likely` and `unlikely` may be defined as function-like macros **and may be undefined**.

3055. Misleading body for surrogate call function

Section: 12.2.2.2.3 [[over.call.object](#)] **Status:** ready **Submitter:** Brian Bi **Date:** 2025-08-11

(From submission [#747](#).)

Subclause 12.2.2.2.3 [[over.call.object](#)] paragraph 2 defines the surrogate call function has having a function body. However, this body will never be evaluated, because 12.4.4 [[over.call](#)] paragraph 1 specifies the behavior when a surrogate call function is selected by overload resolution.

Proposed resolution (approved by CWG 2025-11-05):

Change in 12.2.2.2.3 [[over.call.object](#)] paragraph 2 as follows:

..., a *surrogate call function* with the unique name *call-function* and having **a declaration of** the form

```
R call-function ( conversion-type-id F, P1 a1, ... , Pn an) { return F (a1, ..., an); } ;
```

is also considered as a candidate function. **[Note: If a surrogate call function is selected by overload resolution, the behavior is as described in 12.4.4 [[over.call](#)]. -- end note]** Similarly,...

3056. Missing semicolons in grammar for *type-requirement*

Section: 7.5.8.3 [[expr.prim.req.type](#)] **Status:** ready **Submitter:** Peter Bindels **Date:** 2025-08-14

(From submission [#751](#).)

These lines in the example in 7.5.8.3 [[expr.prim.req.type](#)] paragraph 1 are not covered by the grammar:

```
typename [:T::r1:];           // fails if T::r1 is not a reflection of a type
typename [:T::r2:<int>];       // fails if T::r2 is not a reflection of a template Z for which Z<int> is a type
```

Proposed resolution (approved by CWG 2025-11-05):

Change in 7.5.8.3 [[expr.prim.req.type](#)] paragraph 1 as follows:

```
type-requirement:
  typename nested-name-specifieropt type-name ;
  typename splice-specifier ;
  typename splice-specialization-specifier ;
```

3057. Ranking of derived-to-base conversions should ignore reference binding

Section: 12.2.4.2.5 [[over.ics.ref](#)] **Status:** ready **Submitter:** Evan Girardin **Date:** 2025-07-31

(From submission [#733](#).)

The derived-to-base tiebreaker rules in 12.2.4.3 [[over.ics.rank](#)] paragraph 4 should apply regardless of whether one destination type is a reference and the other one is not. Major implementations agree.

Proposed resolution (approved by CWG 2025-11-05):

1. Change in 12.2.4.2.5 [[over.ics.ref](#)] paragraph 1 as follows:

When a parameter of type “reference to cv T” binds directly (9.5.4 [[dcl.init.ref](#)]) to an argument expression:

- If the argument expression has a type **D** that is a derived class of the parameter type **T**, the implicit conversion sequence is a derived-to-base conversion **from D to T** (12.2.4.2 [[over.best.ics](#)]).
- ...

2. Change in 12.2.4.3 [[over.ics.rank](#)] bullet 4.5 as follows:

- ...
- If class B is derived directly or indirectly from class A and class C is derived directly or indirectly from B,
 - conversion of C* to B* is better than conversion of C* to A*, [*Example 10: ... —end example*]
 - binding of an expression of type C to a reference to type B is better than binding an expression of type C to a reference to type A;
 - conversion of A::* to B::* is better than conversion of A::* to C::*,
 - conversion of C to B is better than conversion of C to A,
 - conversion of B* to A* is better than conversion of C* to A*,
 - binding of an expression of type B to a reference to type A is better than binding an expression of type C to a reference to type A;
 - conversion of B::* to C::* is better than conversion of A::* to C::*; and
 - conversion of B to A is better than conversion of C to A.

[Note: ...]

3059. throw; in constant expressions

Section: 7.7 [[expr.const](#)] **Status:** ready **Submitter:** Lénárd Szolnoki **Date:** 2025-08-28

(From submission [#756](#).)

Consider:

```
int main() {
    try {
        throw 0;
    } catch (...) {
        constexpr int x = [] {
            try {
                throw;           // #1
            } catch (...) {
                return 1;
            }
        }();
    }
}
```

According to 7.6.18 [[expr.throw](#)] paragraph 3, #1 rethrows the currently handled exception as defined in 14.4 [[except.handle](#)] paragraph 10, which would rethrow the runtime exception inside a constant expression.

Proposed resolution (approved by CWG 2025-11-05):

Add a new bullet after 7.7 [[expr.const](#)] bullet 10.22 as follows:

- ...
- a construction of an exception object, unless the exception object and all of its implicit copies created by invocations of `std::current_exception` or `std::rethrow_exception` (17.9.7 [[propagation](#)]) are destroyed within the evaluation of E;

- a *throw-expression* (7.6.18 [[expr.throw](#)]) with no operand, unless there is a currently handled exception whose exception object was constructed within the evaluation of E;
- ...

3060. Change in behavior for `noexcept` main

Section: 6.10.3.1 [[basic.start.main](#)] **Status:** ready **Submitter:** Jiang An **Date:** 2025-08-15

(From submission [#752](#).)

Consider:

```
int main() noexcept {}
```

Prior to P0012R1 (C++17), this was allowed, because `noexcept` was not part of a function type. This is now ill-formed, because such a `main` function is not of one of the types allowed in 6.10.3.1 [[basic.start.main](#)] paragraph 2.

This needs an Annex C entry or a relaxation of the rules. Note that the presence or absence of `noexcept` does not change behavior, because an exception leaving `main` calls `std::terminate` either way, and `main` cannot be named in expressions.

Proposed resolution (approved by CWG 2025-11-05):

Change in 6.10.3.1 [[basic.start.main](#)] paragraph 2 as follows:

... An implementation shall allow both

- an "optionally `noexcept`" function of () returning `int`" and
- an "optionally `noexcept`" function of (int, pointer to pointer to char) returning `int`"

as the type of `main` (9.3.4.6 [[decl.fct](#)]). ...

3061. Trailing comma in an *expansion-init-list*

Section: 8.7 [[stmt.expand](#)] **Status:** ready **Submitter:** Jan Schultke **Date:** 2025-08-24

(From submission [#754](#).)

Brace-enclosed lists generally allow trailing commas. But an *expansion-init-list* does not; consider:

```
for (int x : { 1, }) { }    // OK
template for (int x : { 1, }) { }    // syntax error
```

Proposed resolution (approved by CWG 2025-11-06):

Change in 8.7 [[stmt.expand](#)] paragraph 1 as follows:

```
expansion-init-list :
{ expression-listopt }
{ expression-listopt }
{ }
```

3062. Overlapping specification of default template arguments

Section: 9.3.4.7 [[decl.fct.default](#)] **Status:** ready **Submitter:** Brian Bi **Date:** 2025-08-13

(From submission [#750](#).)

As written, 9.3.4.7 [dcl.fct.default] applies to default arguments for constant template parameters, which are specified in 13.2 [temp.param] and 13.4.3 [temp.arg.nontype]. Remove the overlap.

Proposed resolution (approved by CWG 2025-11-06):

1. Change in 9.3.4.7 [dcl.fct.default] paragraph 1 as follows:

If an *initializer-clause* is specified in a *parameter-declaration* **that is not a template-parameter (13.2 [temp.param])**, this *initializer-clause* is used as a default argument. [Note 1: Default arguments will be used in calls where trailing arguments are missing (7.6.1.3 [expr.call]). — end note]

2. Change in 9.3.4.7 [dcl.fct.default] paragraph 3 as follows:

A default argument shall be specified only in the *parameter-declaration-clause* of a function declaration or *lambda-declarator* ~~or in a template-parameter (13.2 [temp.param])~~. A default argument shall not be specified for ~~a template parameter pack or~~ a function parameter pack. ~~If it is specified in a parameter-declaration-clause, it~~ **A default argument** shall not occur within a *declarator* or *abstract-declarator* of a *parameter-declaration*. [Footnote:]

3. Change in 13.2 [temp.param] paragraph 17 as follows:

A default template argument is a template argument (13.4 [temp.arg]) specified after = in a *template-parameter*. A default template argument ~~may be specified for any kind of template parameter that is not~~ **shall not be specified for** a template parameter pack (13.7.4 [temp.variadic]). A default template argument may be specified in a template declaration. ...

3063. Lifetime extension of temporaries past function return

Section: 6.8.7 [class.temporary] **Status:** ready **Submitter:** Hubert Tong **Date:** 2025-09-12 [N5028 comment CA 040](#)

(From submission [#760](#).)

Consider:

```
struct B { ~B(); };
struct A { const B &b; };
A foo() { return {{}}; }    // #1
void bar();
int main() {
    A a = foo();
    bar();
}
```

At #1, a temporary of type B is created and, due to guaranteed copy elision, is bound to a.b in main. The current rules, as amended by [P2748R5](#) (Disallow Binding a Returned Glvalue to a Temporary), prescribe lifetime extension of that temporary, which is a novel requirement accidentally imposed by P2748R5 that is hard to implement.

Suggested resolution (Option 1) [SUPERSEDED]:

Add in 6.8.7 [class.temporary] bullet 6.11 as follows:

- ...
- A temporary object bound to a reference element of an aggregate of class type initialized from a parenthesized *expression-list* (9.5 [dcl.init]) persists until the completion of the full-expression containing the *expression-list*.
- The lifetime of a temporary bound to **a constituent reference (7.7 [expr.const])** of the returned object in a function return statement (8.8.4 [stmt.return]) is not extended; the temporary is destroyed at the end of the full-expression in the return statement.
- A temporary bound to a reference in a *new-initializer* (7.6.2.8 [expr.new]) persists until ...

Proposed resolution (approved by CWG 2025-11-04):

1. Move to before 6.8.2 [intro.object] paragraph 11 from 7.7 [expr.const] paragraph 2:

The constituent values of an object o are

- if o has scalar type, the value of o;
- otherwise, the constituent values of any direct subobjects of o other than inactive union members.

The constituent references of an object o are

- any direct members of o that have reference type, and

- the constituent references of any direct subobjects of *o* other than inactive union members.

Some operations are described as implicitly creating objects ...

2. Remove 7.7 [expr.const] paragraph 2 and change paragraph 3 as follows:

~~The constituent values of an object *o* are~~
~~◦ if *o* has scalar type, the value of *o*;~~
~~◦ otherwise, the constituent values of any direct subobjects of *o* other than inactive union members.~~
~~The constituent references of an object *o* are~~
~~◦ any direct members of *o* that have reference type, and~~
~~◦ the constituent references of any direct subobjects of *o* other than inactive union members.~~

The constituent values and constituent references of a variable *x* are defined as follows:

- If *x* declares an object, the constituent values and references of that object (6.8.2 [intro.object]) are constituent values and references of *x*.
- If *x* declares a reference, that reference is a constituent reference of *x*.

...

3. Add in 8.8.4 [stmt.return] paragraph 6 as follows:

In a function ~~whose return type is a reference,~~ other than an invented function for std::is_convertible (21.3.8 [meta.rel]), a return statement that binds

- ~~the~~ **a** returned reference **or**
- **a** constituent reference (6.8.2 [intro.object]) of a returned object

to a temporary expression (6.8.7 [class.temporary]) is ill-formed. [Example: ...]

3066. Declarative *nested-name-specifier* in explicit instantiation

Section: 7.5.5.3 [expr.prim.id.qual] **Status:** ready **Submitter:** Jens Maurer **Date:** 2025-03-14

Explicit instantiations with a *nested-name-specifier* do not seem to be declarative, but should be.

Proposed resolution (approved by CWG 2025-09-12):

Change in 7.5.5.3 [expr.prim.id.qual] paragraph 2 as follows:

A *nested-name-specifier* is declarative if it is part of

- a *class-head-name* (11.1 [class.pre]),
- an *enum-head-name* (9.8.1 [dcl.enum]),
- a *qualified-id* that is the *id-expression* of a *declarator-id* (9.3.1 [dcl.decl.general]), **or**
- **an *elaborated-type-specifier* of an explicit instantiation (13.9.3 [temp.explicit]), or**
- a declarative *nested-name-specifier*.

3067. Array-to-pointer conversion with object type mismatch

Section: 7.3.3 [conv.array] **Status:** ready **Submitter:** Andrey Erokhin **Date:** 2020-10-28

(From submission #707.)

Consider:

```
int a[5];
int *p1 = &a[0];           // points to the first element of a
auto pa = reinterpret_cast<int(*)[5]>(p1);
int *p2 = *pa;              // valid array-to-pointer conversion?
```

If a glvalue of array type is subject to the array-to-pointer conversion, the behavior should be undefined if the type of the result of the glvalue is not an array (see 7.6.1.5 [expr.ref] paragraph 10 for the class member access case).

Proposed resolution (approved by CWG 2025-11-05):

Change in 7.3.3 [\[conv.array\]](#) as follows:

An lvalue or rvalue expression *E* of type “array of N T” or “array of unknown bound of T” can be converted to a prvalue of type “pointer to T”. The If *E* is a prvalue, the temporary materialization conversion (7.3.5 [\[conv.rval\]](#)) is applied. The If the result of *E* (possibly converted) is an object whose type is similar to the type of *E*, the result is a pointer to the first element of the array; otherwise, the behavior is undefined.

3070. Trivial assignment can skip member subobjects

Section: 11.4.6 [\[class.copy.assign\]](#) **Status:** ready **Submitter:** Jiang An **Date:** 2025-09-05

(From submission [#759](#).)

Consider:

```
struct B0 { int b0; };

struct B {
    B &operator=(const B &) = default;
    int x;
};

struct D : B0, B {
    using B::operator=;
private:
    D &operator=(const D &) && = default;
};

struct Q {
    Q &operator=(const Q &) = default;
    D d;
};
```

According to the rules, `Q::operator=` is trivial, but it does not copy the `d.B0::b0` member of `Q`. Implementations disagree and copy that member regardless.

We can make `Q::operator=` non-trivial or deleted. We can also make that assignment operator copy all subobjects, even though overload resolution on the base classes clearly is at odds with that outcome.

Proposed resolution (approved 2025-11-05):

1. Change in 11.4.5.3 [\[class.copy.ctor\]](#) paragraph 11 as follows:

- A copy/move constructor for class *X* is trivial if it is not user-provided and if
- class *X* has no virtual functions (11.7.3 [\[class.virtual\]](#)) and no virtual base classes (11.7.2 [\[class.mi\]](#)), and
 - the constructor selected to copy/move each direct base class subobject is a direct member of that base class and is trivial, and
 - for each non-static data member of *X* that is of class type (or array thereof), the constructor selected to copy/move that member is trivial;
- otherwise the copy/move constructor is non-trivial.

2. Change in 11.4.6 [\[class.copy.assign\]](#) paragraph 9 as follows:

- A copy/move assignment operator for class *X* is trivial if it is not user-provided and if
- class *X* has no virtual functions (11.7.3 [\[class.virtual\]](#)) and no virtual base classes (11.7.2 [\[class.mi\]](#)), and
 - the assignment operator selected to copy/move each direct base class subobject is a direct member of that base class and is trivial, and
 - for each non-static data member of *X* that is of class type (or array thereof), the assignment operator selected to copy/move that member is trivial;
- otherwise the copy/move assignment operator is non-trivial.

3071. Negative tuple_size in structured bindings

Section: 9.7 [\[decl.struct.bind\]](#) **Status:** ready **Submitter:** Corentin Jabot **Date:** 2025-09-18

(From submission [#764](#).)

Consider:

```
namespace std {
    template <typename T> struct tuple_size;
}

struct S {
    int a;
};

template <> struct std::tuple_size<S> {
    static constexpr int value = -1;
};

void f(auto) { auto [... a] = S{}; }
```

There is no protection against negative `tuple_size`.

Proposed resolution (approved by CWG 2025-11-04):

Change in 9.7 [[dcl.struct.bind](#)] paragraph 7:

Otherwise, if the *qualified-id* `std::tuple_size<E>` names a complete class type with a member named `value`, the expression `std::tuple_size<E>::value` shall be a well-formed integral constant expression **whose value is non-negative**; **and** the structured binding size of `E` is equal to **the** **that** value **of that expression**. ...

3072. Incorrect examples for lambda SFINAE

Section: 13.10.3.1 [[temp.deduct.general](#)] **Status:** ready **Submitter:** Brian Bi **Date:** 2025-09-13

(From submission [#761](#).)

After issue 2672, the third and fourth comments in Example 7 in 13.10.3.1 [[temp.deduct.general](#)] paragraph 9 incorrectly treat the *lambda-introducer* and trailing return type of a lambda as if they were part of the lambda body.

Proposed resolution (approved by CWG 2025-11-04):

Change in 13.10.3.1 [[temp.deduct.general](#)] paragraph 9 as follows:

[Example: ...

```
template <class T>
    auto h(T) -> decltype([x = T::invalid]() { });
void h(...);
h(0);          // error: invalid expression not part of the immediate context OK, calls h(...)

template <class T>
    auto i(T) -> decltype([]() -> typename T::invalid { });
void i(...);
i(0);          // error: invalid expression not part of the immediate context OK, calls i(...)

template <class T>
    auto j(T t) -> decltype([](auto x) -> decltype(x.invalid) { } (t)); // #1
void j(...);    // #2

j(0);          // deduction fails on #1; OK, calls #2 j(...)

-- end example ]
```

3073. Dependence of *R* on *T2* is unclear

Section: 12.2.2.7 [[over.match.ref](#)] **Status:** ready **Submitter:** Brian Bi **Date:** 2025-09-15

(From submission [#762](#).)

It is unclear whether the set R defined in 12.2.2.7 [[over.match.ref](#)] bullet 1.1 is a function of T_2 or is the union of the sets computed from all such T_2 .

Proposed resolution (approved by CWG 2025-11-04):

Remove the top-level bulleting in 12.2.2.7 [[over.match.ref](#)] bullet 1.1 and change as follows:

... Let R be ~~a~~ **the** set of **all** types including

- ~~“lvalue reference to cv2 T_2 ”~~ **lvalue reference types** (when converting to an lvalue) and
- ~~“cv2 T_2 ” and “rvalue reference to cv2 T_2 ”~~ **non-reference types and rvalue reference types** (when converting to an rvalue or an lvalue of function type)

~~for any T_2 . The permissible types for non-explicit conversion functions are the members of R~~ **having the form “cv T_2 ” or “reference to cv2 T_2 ”** where “cv1 T ” is reference-compatible (9.5.4 [[dcl.init.ref](#)]) with “cv2 T_2 ”. For direct-initialization, the permissible types for explicit conversion functions are the members of R **having the form “cv2 T_2 ” or “reference to cv2 T_2 ”** where T_2 can be converted to type T with a (possibly trivial) qualification conversion (7.3.6 [[conv.qual](#)]); otherwise there are none.

3074. Redundant ill-formedness for module macros

Section: 15.5 [[cpp.module](#)] **Status:** ready **Submitter:** Hubert Tong **Date:** 2025-09-21

(From submission [#766](#).)

Macro definitions for keywords are prohibited by 15.7.1 [[cpp.replace.general](#)] paragraph 9. More limited restrictions in 15.5 [[cpp.module](#)] paragraph 1 and 15.6 [[cpp.import](#)] paragraph 1 are redundant.

Proposed resolution (approved by CWG 2025-09-26):

1. Remove 15.5 [[cpp.module](#)] paragraph 1:

~~A *pp-module* shall not appear in a context where module or (if it is the first preprocessing token of the *pp-module*) export is an identifier defined as an object-like macro.~~

2. Remove 15.6 [[cpp.import](#)] paragraph 1:

~~A *pp-import* shall not appear in a context where import or (if it is the first preprocessing token of the *pp-import*) export is an identifier defined as an object-like macro.~~

3075. Unclear matching of import directive

Section: 15.6 [[cpp.import](#)] **Status:** ready **Submitter:** Hubert Tong **Date:** 2025-09-22

(From submission [#767](#).)

Subclause 15.6 [[cpp.import](#)] paragraph 2 is unclear whether the matching of the last form happens before or after macro replacement.

Proposed resolution (approved by CWG 2025-09-26):

Change in 15.6 [[cpp.import](#)] paragraph 2 as follows:

The preprocessing tokens after the import preprocessing token in the import *control-line* are processed just as in normal text (i.e., each identifier currently defined as a macro name is replaced by its replacement list of preprocessing tokens). [*Note 1*: An import directive matching the first two forms of a *pp-import* instructs the preprocessor to import macros from the header unit (10.3 [[module.import](#)]) denoted by the *header-name*, as described below. — *end note*] The point of macro import for the first two forms of *pp-import* is immediately after the *new-line* terminating the *pp-import*. The last form of *pp-import* **does not have a point of macro import, and** is only considered if, **after macro replacement**, the first two forms did not match, ~~and does not have a point of macro import.~~

3076. Remove unnecessary IFNDR for malformed *header-name-tokens*

Section: 15.3 [[cpp.include](#)] **Status:** ready **Submitter:** Hubert Tong **Date:** 2025-09-22

(From submission [#768](#).)

Consider:

```
#include L"hello"
```

When forming a *header-name* preprocessing token from *pp-tokens* in 15.3 [[cpp.include](#)] paragraph 7 is bound to fail when the first such token is a *string-literal* with an *encoding-prefix* or R. Such a failure is specified to be ill-formed, no diagnostic required, but implementations uniformly diagnose the situation.

Proposed resolution (approved by CWG 2025-09-26):

1. Change in 5.13.5 [[lex.string](#)] paragraph 1 as follows:

```
string-literal:  
  encoding-prefixopt " s-char-sequenceopt " plain-string-literal  
  encoding-prefixopt R raw-string
```

```
plain-string-literal:  
  " s-char-sequenceopt "
```

2. Change in 15.3 [[cpp.include](#)] paragraph 7 as follows:

A preprocessing directive of the form

```
# include pp-tokens new-line
```

(that does not match the previous form) is permitted. The preprocessing tokens after `include` in the directive are processed just as in normal text (i.e., each identifier currently defined as a macro name is replaced by its replacement list of preprocessing tokens). **After replacement, if the first preprocessing token is a *string-literal*, it shall be a *plain-string-literal*.** Then, an attempt is made to form a *header-name* preprocessing token (5.6 [[lex.header](#)]) from the whitespace and the characters of the spellings of the resulting sequence of preprocessing tokens; the treatment of whitespace is implementation-defined. If the attempt succeeds, the directive with the so-formed *header-name* is processed as specified for the previous form. Otherwise, the program is ill-formed, no diagnostic required.

Editing note: The second change of the proposed resolution is superseded by the resolution of issue 3078.

3077. Undesirable formation of `import` directive with *string-literal*

Section: 15.1 [[cpp.pre](#)] **Status:** ready **Submitter:** Hubert Tong **Date:** 2025-09-22

(From submission [#769](#).)

Consider:

```
#define STR(X) #X  
const char *str = STR(  
  import u8"hello";    // #1  
);
```

Line #1 is recognized as an `import` directive, even though the eventual attempt to form a *header-name* will be IFNDR (see issue 3076). This is incongruent with the stated design goal of P1857: since a *header-name* is lexed right away, any *string-literals* that remain are necessarily invalid.

Proposed resolution (approved by CWG 2025-09-26):

Change in 15.1 [[cpp.pre](#)] bullet 1.2 as follows:

- ...

- an import preprocessing token immediately followed on the same logical source line by a *header-name*, *<*, *identifier*, *string-literal*, or *:* preprocessing token, or
- ...

3078. Different treatment of `#include` *pp-tokens* and *header-name-tokens*

Section: 15.3 [[cpp.include](#)] **Status:** ready **Submitter:** Hubert Tong **Date:** 2025-09-22

(From submission [#770](#).)

Consider:

```
#define X >
#include <<X
```

As further clarified by issue 3015, this performs the same inclusion as

```
#include <<>
```

There is implementation divergence; clang accepts; GCC, EDG, and MSVC reject.

There are related concerns when the character sequence of a digraph appears in prospective *header-name*. The following is ill-formed because `<%` is a digraph:

```
#define X >
#if __has_include(<%X)
#endif
```

However, the same character sequence is valid in `#include`:

```
#define X >
#include <%X    // valid, includes %
```

Thus the footnote in 5.9 [[lex.digraph](#)] paragraph 2 is overly broad.

Proposed resolution (approved by CWG 2025-11-04):

1. Change in 5.9 [[lex.digraph](#)] paragraph 2 as follows:

In all respects of the language, each alternative token behaves the same, respectively, as its primary token, except for its spelling. [~~Footnote: Thus the~~ **Note: The** “stringized” values (15.7.3 [[cpp.stringize](#)]) of `[` and `<`: ~~will be~~ **are** different, maintaining the source spelling, ~~but the tokens can otherwise be freely interchanged.~~] The set of alternative tokens is defined in Table 3.

2. Change in 15.2 [[cpp.cond](#)] paragraph 1 as follows:

```
header-name-tokens:
  string-literal plain-string-literal
  < h-pp-tokens >
```

3. Change in 15.3 [[cpp.include](#)] paragraph 7 (supersedes the change to that paragraph from issue 3076):

A preprocessing directive of the form

```
# include pp-tokens new-line
```

(that does not match the previous form) is permitted. The preprocessing tokens after `include` in the directive are processed just as in normal text (i.e., each identifier currently defined as a macro name is replaced by its replacement list of preprocessing tokens). **The resulting sequence of preprocessing tokens shall be of the form**

```
header-name-tokens
```

~~Then, an~~ **An** attempt is **then** made to form a *header-name* preprocessing token (5.6 [[lex.header](#)]) from the whitespace and the characters of the spellings of the ~~resulting sequence of preprocessing tokens~~ **header-name-tokens**; the treatment of whitespace is implementation-defined. If the attempt succeeds, the directive with the so-formed *header-name* is processed as specified for the previous form. Otherwise, the program is ill-formed, no diagnostic required.

Note: The third change of the resolution supersedes the second change in the resolution of issue 3076.

3079. Allow *empty-declarations* in anonymous unions

Section: 11.5.2 [[class.union.anon](#)] **Status:** ready **Submitter:** keinflue **Date:** 2025-08-27

(From submission [#755](#).)

Subclause 11.5.2 [[class.union.anon](#)] paragraph 1 specifies:

... Each *member-declaration* in the *member-specification* of an anonymous union shall either define one or more public non-static data members or be a *static_assert-declaration*. Nested types, anonymous unions, and functions shall not be declared within an anonymous union. ...

This rule yields:

```
struct A { union {int x;;} u; };    // OK
struct B { union {int y;;}; };      // error
```

Both class definitions ought to be allowed. There is implementation divergence: MSVC, gcc, and EDG accept both class definition; clang accepts both in default mode and rejects both in pedantic mode (rejecting empty declarations in all class definitions).

Proposed resolution (approved by CWG 2025-11-04):

Change in 11.5.2 [[class.union.anon](#)] paragraph 1 as follows:

... Each *member-declaration* in the *member-specification* of an anonymous union shall ~~either~~ define one or more public non-static data members, **be an *empty-declaration***, or be a *static_assert-declaration*. Nested types, anonymous unions, and functions shall not be declared within an anonymous union. ...

3080. Clarify kinds of permitted template template arguments

Section: 13.4.4 [[temp.arg.template](#)] **Status:** ready **Submitter:** CWG **Date:** 2025-09-26

(Split off from issue 3003.)

The specification in 13.4.4 [[temp.arg.template](#)] paragraph 1 omits template template parameters from the list of entities that can be used as template arguments. This is unintentional.

Proposed resolution (approved by CWG 2025-09-26):

Change in 13.4.4 [[temp.arg.template](#)] paragraph 1 as follows:

A *template-argument* for a template template parameter shall be the name of a template. For a *type-tt-parameter*, the name shall denote a class template ~~or~~ , alias template, **or type template template parameter**. For a *variable-tt-parameter*, the name shall denote a variable template **or variable template template parameter**. For a *concept-tt-parameter*, the name shall denote a concept **or concept template parameter**. ...

3081. Require glvalue when splicing direct base class relationship

Section: 7.6.1.5 [[expr.ref](#)] **Status:** ready **Submitter:** US **Date:** 2025-10-01 [N5028 comment US 29-062](#)

In a class member access, if the second expression represents a direct base class relationship, the first expression should be required to be a glvalue.

Possible resolution (reviewed by CWG 2025-10-10). [SUPERSEDED]:

Change in 7.6.1.5 [[expr.ref](#)] paragraph 2 as follows:

For a dot that is followed by an expression that designates a static member (11.4.9.3 [class.static.data]) or an enumerator (9.8.1 [dcl.enum]), the first expression is a discarded-value expression (7.2.3 [expr.context]); if the expression after the dot designates a non-static data member (11.4.1 [class.mem.general]) or a direct base class relationship (11.7.1 [class.derived.general]), the first expression shall be a glvalue. A postfix expression that is followed by an arrow shall be a prvalue having pointer type.

Proposed resolution (approved by CWG 2025-11-03):

1. Change in 6.8.7 [class.temporary] bullet 6.4 as follows:

- o ...
- o a class member access (7.6.1.5 [expr.ref]) using the . operator where the left operand is one of these expressions and the right operand designates a non-static data member (11.4.1 [class.mem.general]) of non-reference type or a direct base class relationship (11.7.1 [class.derived.general]),
- o ...

2. Change in 7.6.1.5 [expr.ref] paragraph 2 as follows:

For a dot that is followed by an expression that designates a static member or an enumerator, the first expression is a discarded-value expression (7.2.3 [expr.context]); if the expression after the dot designates a non-static data member (11.4.1 [class.mem.general]) or a direct base class relationship (11.7.1 [class.derived.general]), the first expression shall be a glvalue. A postfix expression that is followed by an arrow shall be a prvalue having pointer type.

3082. Allow for call-compatible function types in reinterpret_cast

Section: 7.6.1.10 [expr.reinterpret.cast] **Status:** ready **Submitter:** US **Date:** 2025-10-01 [N5028 comment US 30-061](#)

Subclause 7.6.1.10 [expr.reinterpret.cast] paragraph 6 does not, but should, make casts to call-compatible function types predictably valid.

Proposed resolution (approved by CWG 2025-10-10):

Change in 7.6.1.10 [expr.reinterpret.cast] paragraph 6 as follows:

A function pointer can be explicitly converted to a function pointer of a different type. **The function pointer value is unchanged by the conversion.** [Note 4: The effect of calling a function through a pointer to a function type (9.3.4.6 [dcl.fct]) that is not the same as call-compatible with the type used in the definition of the function is undefined (7.6.1.3 [expr.call]). — end note] Except that converting a prvalue of type “pointer to T1” to the type “pointer to T2” (where T1 and T2 are function types) and back to its original type yields the original pointer value, the result of such a pointer conversion is unspecified.

3083. Remove redundant restrictions on class and enum definitions

Section: 8.1 [stmt.pre] **Status:** ready **Submitter:** US **Date:** 2025-10-01 [N5028 comment US 35-066](#)

Subclause 8.1 [stmt.pre] paragraph 8 has the following redundant specification:

In the *decl-specifier-seq* of a condition or of a *for-range-declaration*, including that of any *structured-binding-declaration* of the condition, each *decl-specifier* shall be either a *type-specifier* or *constexpr*. The *decl-specifier-seq* of a *for-range-declaration* shall not define a class or enumeration.

The second sentence is redundant, because a *type-specifier* (as opposed to a *defining-type-specifier*) cannot define a class or enumeration.

Proposed resolution (approved by CWG 2025-10-24):

Change in 8.1 [stmt.pre] paragraph 8 as follows:

In the *decl-specifier-seq* of a condition or of a *for-range-declaration*, including that of any *structured-binding-declaration* of the condition, each *decl-specifier* shall be either a *type-specifier* or *constexpr*. ~~The *decl-specifier-seq* of a *for-range-declaration* shall not define a class or enumeration.~~

3084. *compound-statements* inside *iteration-statements*

Section: 8.8.3 [[stmt.cont](#)] **Status:** ready **Submitter:** US **Date:** 2025-10-01 [N5028 comment US 36-069](#)

There is no *iteration-statement* production that directly contains a *compound-statement*.

Proposed resolution (approved by CWG 2025-10-24):

Change in 8.8.3 [[stmt.cont](#)] paragraph 1 as follows:

A *continue* statement shall be enclosed by (8.1 [[stmt.pre](#)]) an *iteration-statement* or an *expansion-statement*. If the innermost enclosing such statement X is an *iteration-statement* (8.6 [[stmt.iter](#)]), the *continue* statement causes control to pass to the end of the *statement* ~~or *compound-statement*~~ of X. Otherwise, control passes to the end of the *compound-statement* of the current Si (8.7 [[stmt.expand](#)]).

3085. Apply restriction inside *for-range-declaration*

Section: 8.1 [[stmt.pre](#)] **Status:** ready **Submitter:** US **Date:** 2025-10-01 [N5028 comment US 34-067](#)

The restriction in 8.1 [[stmt.pre](#)] paragraph 1 should also apply to any *structured-binding-declaration* in a *for-range-declaration*.

Proposed resolution (approved by CWG 2025-11-06):

Change in 8.1 [[stmt.pre](#)] paragraph 8 as follows:

Let *D* be any *condition* or *for-range-declaration*. In the *decl-specifier-seq* of ~~a *condition* or of a *for-range-declaration*~~ ***D***, including that of any *structured-binding-declaration* of ~~the *condition*~~ ***D***, each *decl-specifier* shall be either a *type-specifier* or *constexpr*.

3086. Destringizing should consider all sorts of *encoding-prefixes*

Section: 15.13 [[cpp.pragma.op](#)] **Status:** ready **Submitter:** US **Date:** 2025-10-01 [N5028 comment US 60-110](#)

Subclause 15.13 [[cpp.pragma.op](#)] paragraph 1 specifies:

A unary operator expression of the form:

```
_Pragma ( string-literal )
```

is processed as follows: The *string-literal* is destringized by deleting the L prefix, if present, deleting the leading and trailing double-quotes, replacing each escape sequence \" by a double-quote, and replacing each escape sequence \\ by a single backslash. The resulting sequence of characters is processed through translation phase 3 to produce preprocessing tokens that are executed as if they were the *pp-tokens* in a *pragma* directive. The original four preprocessing tokens in the unary operator expression are removed.

The destringizing does not, but should, consider *encoding-prefixes* other than L.

See also C23 6.10.11.

Proposed resolution (approved by CWG 2025-10-24):

Change in 15.13 [[cpp.pragma.op](#)] paragraph 1 as follows:

A unary operator expression of the ~~form:~~ **form**

```
_Pragma ( string-literal )
```

is processed as follows: The *string-literal* is destringized by deleting ~~the L prefix, if present~~ **any *encoding-prefix***, deleting the leading and trailing double-quotes, replacing each escape sequence \" by a double-quote, and replacing each escape sequence \\ by a single backslash. The resulting sequence of characters is processed through translation phase 3 to produce preprocessing tokens that are executed as if they were the *pp-tokens* in a *pragma* directive. The original four preprocessing tokens in the unary operator expression are removed.

3089. const-default-constructible improperly handles std::meta::info

Section: 9.5.1 [[dcl.init.general](#)] **Status:** ready **Submitter:** US **Date:** 2025-10-01 [N5028 comment US 39-076](#)

A class that contains only std::meta::info data members, or arrays thereof, should be const-default-constructible.

Proposed resolution (approved by CWG 2025-10-24):

Change in 9.5.1 [[dcl.init.general](#)] paragraph 8 as follows (add bullets):

A ~~class~~ type `cv T` is *const-default-constructible* if

- `T` is `std::meta::info`;
- `T` is `std::nullptr_t`;
- default-initialization of `T` would invoke a user-provided constructor of `T` (not inherited from a base class);
- ~~or if~~ **`T` is a class type where**
 - each direct non-variant non-static data member `M` of `T` has a default member initializer or, if `M` is of ~~class type~~ `X` (or array thereof), `X` is const-default-constructible **type**,
 - if `T` is a union with at least one non-static data member, exactly one variant member has a default member initializer,
 - if `T` is not a union, ~~for the type of~~ **each anonymous union member is const-default-constructible** ~~with at least one non-static data member (if any), exactly one non-static data member has a default member initializer, and~~
 - each potentially constructed base class of `T` is const-default-constructible; **or**
- `T` is an array of const-default-constructible type.

If a program calls for the default-initialization of an object of a const-qualified type `T`, `T` shall be ~~std::meta::info or~~ a const-default-constructible ~~class type, or array thereof~~.

3090. Internal linkage from header units

Section: 10.2 [[module.interface](#)] **Status:** ready **Submitter:** US **Date:** 2025-10-01 [N5028 comment US 43-080](#)

The note in 10.2 [[module.interface](#)] paragraph 7 incorrectly omits internal linkage names provided by header units; see 10.3 [[module.import](#)] paragraph 6.

Proposed resolution (approved by CWG 2025-11-06):

Change in 10.2 [[module.interface](#)] paragraph 7 as follows:

[Note 3: Names introduced by exported declarations **never have module linkage. They** have ~~either~~ external linkage ~~or~~, no linkage, **or (in the case of header units) internal linkage**; see 6.7 [[basic.link](#)]. Namespace-scope declarations exported by a module can be found by name lookup in any translation unit importing that module (6.5 [[basic.lookup](#)]). Class and enumeration member names can be found by name lookup in any context in which a definition of the type is reachable. — end note]

3091. Linking of translation units as sequences of tokens

Section: 6.7 [[basic.link](#)] **Status:** ready **Submitter:** US **Date:** 2025-10-01 [N5028 comment US 15-032](#)

Clarify that translated translation units are linked to form a program.

Proposed resolution (approved by CWG 2025-11-04):

1. Change in 5.2 [[lex.phases](#)] paragraph 8 as follows:

8. Translated translation units are combined, and all external entity references are resolved (**6.7 [[basic.link](#)]**). Library components are linked to satisfy external references to entities not defined in the current translation. All such translator output is collected into a program image which contains information needed for execution in its execution environment.

2. Change in 6.7 [\[basic.link\]](#) paragraph 1 as follows:

A *program* consists of one or more translation units (5.1 [\[lex.separate\]](#)) **that are translated and** linked together. A translation unit consists of a sequence of declarations. ...

3092. *base-specifiers* are not "declared"

Section: 9.13.12 [\[decl.attr.annotation\]](#) **Status:** ready **Submitter:** US **Date:** 2025-10-01 [N5028 comment US 41-079](#)

A *base-specifier* is not declared and thus does not have a declaration.

Proposed resolution (approved by CWG 2025-10-24):

Change in 9.13.12 [\[decl.attr.annotation\]](#) paragraph 1 as follows:

An annotation may be applied **to a *base-specifier* or** to any declaration of a type, type alias, variable, function, namespace, enumerator, ~~*base-specifier*~~, or non-static data member.

3093. Missing integration of direct base class relationships

Section: 7.5.9 [\[expr.prim.splice\]](#) **Status:** ready **Submitter:** US **Date:** 2025-10-01 [N5028 comment US 27-059](#)

(From submissions [#777](#) and [#778](#).)

Subclause 7.5.9 [\[expr.prim.splice\]](#) paragraph 2 makes no provision for splicing a reflection representing a direct base class relationship, yet 7.6.1.5 [\[expr.ref\]](#) bullet 8.6 presupposes that this is possible.

Furthermore, 6.3 [\[basic.def.odr\]](#) paragraph 3 does not consider direct base class relationships, nor does 13.8.3.3 [\[temp.dep.expr\]](#) paragraph 5.

Proposed resolution (approved by CWG 2025-11-05):

1. Change in 6.3 [\[basic.def.odr\]](#) bullet 3.3 as follows:

- ...
- If E is a class member access expression (7.6.1.5 [\[expr.ref\]](#)) of the form E1 . template_{opt} E2 ~~naming~~, **where E2 designates** a non-static data member **or a direct base class relationship**, the set contains the potential results of E1.
- ...

2. Change in 7.2.1 [\[basic.lval\]](#) bullet 1.1 as follows:

- A glvalue is an expression whose evaluation determines the identity of an object, function, ~~or~~ non-static data member, **or direct base class relationship**.
- ...

3. Insert a paragraph before 7.5.5.1 [\[expr.prim.id.general\]](#) paragraph 5 as follows:

A splice-expression that designates a direct base class relationship shall appear only as the second operand of a class member access.

For an *id-expression* that denotes an overload set, overload resolution is performed ...

4. Insert before 7.5.9 [\[expr.prim.splice\]](#) bullet 2.4 as follows:

- Otherwise, if S is a direct base class relationship (D, B), the expression is an lvalue designating S. The expression has the type B.
- Otherwise, if S is a variable or a structured binding, ...

5. Change in 7.6.1.5 [\[expr.ref\]](#) paragraph 10 as follows:

If E2 designates a non-static member (possibly after overload resolution) **or direct base class relationship** and the result of E1 is an object whose type is not similar (7.3.6 [[conv.qual](#)]) to the type of E1, the behavior is undefined. ...

6. Change in 12.4.6 [[over.ref](#)] paragraph 1 as follows:

A *class member access operator function* is a function named `operator->` that is a non-static member function taking no non-object parameters. For an expression of the form

`postfix-expression -> templateopt id-expression`

the operator function is selected by overload resolution (12.2.2.3 [[over.match.oper](#)]), and the expression is interpreted as

`( postfix-expression . operator -> () ) -> templateopt id-expression`

Analogously, for an expression of the form

`postfix-expression -> splice-expression`

the operator function is selected by overload resolution, and the expression is interpreted as

`( postfix-expression . operator -> () ) -> splice-expression`

7. Change in 13.8.3.3 [[temp.dep.expr](#)] paragraph 5 as follows:

A class member access expression (7.6.1.5 [[expr.ref](#)]) is type-dependent if

- the terminal name of its *id-expression*, if any, is dependent,
- **its *splice-expression*, if any, is type-dependent**, or
- the expression refers to a member of the current instantiation and the type of the referenced member is dependent.

3094. Rework phases for string literal concatenation and token formation

Section: 5.2 [[lex.phases](#)] **Status:** ready **Submitter:** US **Date:** 2025-10-01 [N5028 comment US 6-020](#)
[N5028 comment US 7-019](#)

Merge phases 5 and 6, because both deal with the same contiguous sequences of string literals. Then, move the conversion of *pp-tokens* to tokens into a new phase 6.

Proposed resolution (approved by CWG 2025-11-04):

1. Change in 5.2 [[lex.phases](#)] paragraph 5 through 7 as follows:

5. For a sequence of two or more adjacent *string-literal* preprocessing tokens, a common *encoding-prefix* is determined as specified in 5.13.5 [[lex.string](#)]. Each such *string-literal* preprocessing token is then considered to have that common *encoding-prefix*. ~~6. Adjacent~~ **Then, adjacent** *string-literal* preprocessing tokens are concatenated (5.13.5 [[lex.string](#)]).

~~7. 6.~~ Each preprocessing token is converted into a token (5.10 [[lex.token](#)]).

~~7.~~ The ~~resulting~~ tokens constitute a translation unit and are syntactically and semantically analyzed as a *translation-unit* (6.7 [[basic.link](#)]) and translated. ...

2. Change in 5.5 [[lex.pptoken](#)] paragraph 1 as follows:

A preprocessing token is the minimal lexical element of the language in translation phases 3 through ~~6~~ **5**.

3. Change in 5.8 [[lex.operators](#)] paragraph 1 as follows:

... Each *operator-or-punctuator* is converted to a single token in translation phase ~~7~~ **6** (5.2 [[lex.phases](#)]).

4. Change in 5.13.5 [[lex.string](#)] paragraph 8 as follows:

In translation phase ~~6~~ **5** (5.2 [[lex.phases](#)]), adjacent *string-literals* are concatenated. The lexical structure and grouping of the contents of the individual *string-literals* is retained.

5. Change in 5.13.9 [[lex.ext](#)] paragraph 8 as follows:

In translation phase ~~6~~ **5** (5.2 [[lex.phases](#)]), adjacent *string-literals* are concatenated and *user-defined-string-literals* are considered *string-literals* for that purpose. During concatenation, *ud-suffix* es are removed and ignored and the concatenation process occurs as described in 5.13.5 [[lex.string](#)]. At the end of phase ~~6~~ **5**, if a *string-literal* is the result of a

concatenation involving at least one *user-defined-string-literal*, all the participating *user-defined-string-literals* shall have the same *ud-suffix* and that suffix is applied to the result of the concatenation.

6. Change in 21.4.16 [[meta.reflection.define.aggregate](#)] bullet 5.2 as follows (addresses alternative tokens (e.g. xor) and exceptions instead of evaluation failure):

Throws: meta::exception unless the following conditions are met:

- ...
- if options.name contains a value, then:
 - holds_alternative<u8string>(options.name->contents) is true and get<u8string>(options.name->contents) contains **the spelling of a valid token that is an identifier** identifier (5.11 [[lex.name](#)]) ~~that is not a keyword (5.12 [[lex.key](#)])~~ when interpreted with UTF-8, or
 - holds_alternative<string>(options.name->contents) is true and get<string>(options.name->contents) contains **the spelling of a valid token that is an identifier** identifier (5.11 [[lex.name](#)]) ~~that is not a keyword (5.12 [[lex.key](#)])~~ when interpreted with the ordinary literal encoding;
- [Note 3: ~~The name corresponds to the spelling of an identifier token after phase 6 of translation (5.2 [[lex.phases](#)]). Lexical constructs like universal-character-names (5.3.2 [[lex.universal.char](#)]) are not processed and will cause evaluation to fail. For example, R"(u03B1)" is an invalid identifier and is not interpreted as "a". — end note]~~
- ...

3095. Type-dependent packs that are not structured binding packs

Section: 13.8.3.3 [[temp.dep.expr](#)] **Status:** ready **Submitter:** AT **Date:** 2025-10-01 [N5028 comment AT 3-098](#)

Subclause 13.8.3.3 [[temp.dep.expr](#)] bullet 3.6 also covers constant template parameter packs with non-dependent type, but those are not type-dependent.

Proposed resolution (approved by CWG 2025-11-03):

Change in 13.8.3.3 [[temp.dep.expr](#)] bullet 3.6 as follows:

- ...
- associated by name lookup with a **structured binding** pack, [Example: ... -- end example]
- ...

3096. Value-dependence of size of structured binding pack with non-dependent initializer

Section: 13.8.3.4 [[temp.dep.constexpr](#)] **Status:** ready **Submitter:** AT **Date:** 2025-10-01 [N5028 comment AT 4-099](#)

The special case for a structured binding pack introduces an unnecessary inconsistency. A structured binding pack with a non-dependent initializer should either always be instantiated early or never, regardless of the expression it is used in. The more comprehensive rule to always instantiate early was removed after R7 of the paper (P1061R7 Structured Bindings can introduce a Pack), but the special case for "sizeof ..." remained.

Proposed resolution (approved by CWG 2025-11-03):

Change in 13.8.3.4 [[temp.dep.constexpr](#)] paragraph 4 as follows:

Expressions of the following form are value-dependent:

```
sizeof ... ( identifier )
fold-expression
```

~~unless the identifier is a structured binding pack whose initializer is not dependent.~~

3097. Lambda expression introduces a scope

Section: 6.4.1 [[basic.scope.scope](#)] **Status:** ready **Submitter:** AT **Date:** 2025-10-01 [N5028 comment AT 4-099](#)

A *lambda-expression* introduces a scope, but it is missing from the list in 6.4.1 [\[basic.scope.scope\]](#) paragraph 1.

Proposed resolution (approved by CWG 2025-10-24):

Change in 6.4.1 [\[basic.scope.scope\]](#) paragraph 1 as follows:

The declarations in a program appear in a number of scopes that are in general discontinuous. The global scope contains the entire program; every other scope *S* is introduced by a declaration, *parameter-declaration-clause*, *statement*, *handler*, *lambda-expression*, or contract assertion (as described in the following subclauses of 6.4 [\[basic.scope\]](#)) appearing in another scope, which thereby contains *S*. An enclosing scope at a program point is any scope that contains it; the smallest such scope is said to be the immediate scope at that point. A scope intervenes between a program point *P* and a scope *S* (that does not contain *P*) if it is or contains *S* but does not contain *P*.

3098. Remove redundancy "names or designates"

Section: 13.3 [\[temp.names\]](#) **Status:** ready **Submitter:** US **Date:** 2025-10-01 [N5028 comment US 50-091](#)

There is expression redundancy (and possible ambiguity) with the phrasing "names or designates" in 13.3 [\[temp.names\]](#) paragraph 8.

Proposed resolution (approved by CWG 2025-10-24):

Change in 13.3 [\[temp.names\]](#) paragraph 8 as follows:

... A *simple-template-id* or *splice-specialization-specifier* shall be valid unless its respective *template-name* or *splice-specifier* **names or** designates a function template (13.10.3 [\[temp.deduct\]](#)). ...

3099. Instantiation of type aliases from alias templates is unspecified

Section: 13.9.2 [\[temp.inst\]](#) **Status:** ready **Submitter:** Hubert Tong **Date:** 2025-10-23

(From submission [#782](#).)

P2996 added the idea that a type alias can result from instantiation of an alias template; however, the timing of the instantiation (and its relation to the immediate context) is unspecified.

Consider:

```
#include <meta>

using namespace std::meta;

template <typename T> using A = T *;

template <auto> struct Sink;

template <typename T> void f(Sink<A<T> > * = 0); // immediately instantiated and considered as part of the immediate context?
template <typename T> void f(int = 0);
void g() { f<int &>(); }

constexpr auto x = substitute(A, {int &}); // valid until dealias?
```

Proposed resolution (approved by CWG 2025-11-04):

Change in 13.7.8 [\[temp.alias\]](#) paragraph 2 as follows:

A

- *template-id* that is not the operand of a *reflect-expression* or
- *splice-specialization-specifier*

that designates the specialization of an alias template is equivalent to the associated type obtained by substitution of its *template-arguments* for the *template-parameter* *s* in the *defining-type-id* of the alias template. Any other *template-id* that names

a specialization of an alias template is a *typedef-name* for a type alias; **such a *template-id* is ill-formed if forming the associated type results in substitution failure.** ...

3100. Destruction order for objects with static storage duration

Section: 6.10.3.4 [[basic.start.term](#)] **Status:** ready **Submitter:** CA **Date:** 2025-10-01 [N5028 comment CA 045](#)

Subclause 6.10.3.4 [[basic.start.term](#)] paragraph 3 conflates the idea of completing a constructor (which, in theory, can include the non-delegating constructor called by a delegating constructor) and completing dynamic initialization of an object (possibly including the destruction of any temporaries). Additional clarity over what happens for block-scope statics initialized as part of executing destructor calls for temporaries of the initialization construct is needed.

Furthermore, the subject paragraph talks about the destructor of objects (of which arrays of class objects have none) and is incompatible with the treatment (consistent with the 2024-11-22 proposed resolution to issue 2929, "Lifetime of trivially-destructible static or thread-local objects") where cleanup registration does not happen individually for the destruction of array elements.

Also, the word "during" in the last sentence of the subject paragraph seems imprecise in the presence of asynchronous execution.

Additionally, without changes like the proposed resolution to issue 2929, that last sentence raises questions as to whether the destruction of subobjects occurs when exit is called during initialization of a complete class object of static storage duration.

Proposed resolution (approved by CWG 2025-11-07):

1. Change in 6.10.3.4 [[basic.start.term](#)] paragraph 3 as follows:

If the ~~completion of the constructor or dynamic initialization~~ **deemed construction (9.5.1 [[dcl.init.general](#)])** of ~~an a~~ **complete** object with static storage duration strongly happens before that of another, the completion of the ~~destructor~~ **destruction** of the second is sequenced before the initiation of the ~~destructor~~ **destruction** of the first. If the ~~completion of the constructor or dynamic initialization~~ **deemed construction** of ~~an a~~ **complete** object with thread storage duration is sequenced before that of another, the completion of the ~~destructor~~ **destruction** of the second is sequenced before the initiation of the ~~destructor~~ **destruction** of the first. If an object is initialized statically, the object is destroyed in the same order as if the object was dynamically initialized. ~~For an object of array or class type, all subobjects of that object are destroyed before any block variable with static storage duration initialized during the construction of the subobjects is destroyed.~~ If the destruction of an object with static or thread storage duration exits via an exception, the function `std::terminate` is called (14.6.2 [[except.terminate](#)]). **[Example: In the following program, the elements of a are destroyed, followed by dt, and finally the two BTemp objects:**

```
struct DTemp { ~DTemp(); };
struct Temp {
    ~Temp() {
        static DTemp dt;
    }
};
struct BTemp {
    ~BTemp();
};
struct A {
    const BTemp &tb;
    ~A();
};
A a[] = { Temp(), BTemp(), BTemp() };

int main() {}
```

If the array a were an object with automatic storage duration, the BTemp temporaries would be destroyed as each element of the array is destroyed (6.8.7 [[class.temporary](#)]). -- end example]

2. Change in 6.10.3.4 [[basic.start.term](#)] paragraph 5 as follows:

If the ~~completion of the initialization~~ **deemed construction** of ~~an a~~ **complete** object with static storage duration strongly happens before a call to `std::atexit` (see `<cstdlib>`, 17.5 [[support.start.term](#)]), the call to the function passed to `std::atexit` is sequenced before the ~~call to the destructor for~~ **initiation of the destruction of** the object. If a call to `std::atexit` strongly happens before the ~~completion of the initialization~~ **deemed construction** of ~~an a~~ **complete** object with static storage duration, the ~~call to the destructor for~~ **completion of the destruction of** the object is sequenced before the call to the function passed to `std::atexit`. If a call to `std::atexit` strongly happens before another call to `std::atexit`, the call to the function passed to the second `std::atexit` call is sequenced before the call to the function passed to the first `std::atexit` call.

3. Change in 9.5.1 [[dcl.init.general](#)] paragraph 22

An object whose initialization has completed. The deemed construction of an object occurs when its initialization completes; for the purposes of 6.10.3.4 [basic.start.term], if the initialization is a full-expression, deemed construction occurs when the evaluation of that full-expression completes. The object is deemed to be constructed, even if the object is of non-class type or no constructor of the object's class is invoked for the initialization. [Note 9: Such an object might have been value-initialized or initialized by aggregate initialization (9.5.2 [dcl.init.aggr]) or by an inherited constructor (11.9.4 [class.inhctor.init]). — end note] Destroying an object of class type invokes the destructor of the class. Destroying a scalar type has no effect other than ending the lifetime of the object (6.8.4 [basic.life]). Destroying an array destroys each element in reverse subscript order.

3101. Types "compounded" from other types

Section: 6.9.1 [basic.types.general] **Status:** ready **Submitter:** Jonathan Wakely **Date:** 2025-10-28

The use of "compounded from" in 6.9.1 [basic.types.general] paragraph 12 is unclear, because the reference to 6.9.4 [basic.compound] appears to include static data member of classes, which seems unintended.

Proposed resolution (approved by CWG 2025-11-07):

Change in 6.9.1 [basic.types.general] paragraph 12 and insert a paragraph as follows:

A type is *consteval-only* if it is either `std::meta::info` or a type compounded from a *consteval-only* type (6.9.4 [basic.compound]).

- `std::meta::info`,
- `cv T`, where `T` is a *consteval-only* type,
- a pointer or reference to a *consteval-only* type,
- an array of *consteval-only* type,
- a function type having a return type or any parameter type that is *consteval-only*,
- a class type with any non-static data member having *consteval-only* type, or
- a type "pointer to member of class `C` of type `T`", where at least one of `C` or `T` is a *consteval-only* type.

Every object of *consteval-only* type shall be ...

3102. Update list of void contexts

Section: 6.9.2 [basic.fundamental] **Status:** ready **Submitter:** Jan Schultke **Date:** 2025-11-02

(From submission #788.)

Subclause 6.9.2 [basic.fundamental] paragraph 15 is missing contexts that yield void. Also, this list ought to be a note.

Proposed resolution (approved by CWG 2025-11-04):

Change in 6.9.2 [basic.fundamental] paragraph 15 as follows:

... [**Note:** An expression of type `cv void` shall **can** be used **only** as

- an expression statement (8.3 [stmt.expr]),
- the expression in a return statement (8.8.4 [stmt.return]) for a function with the return type `cv void`,
- an operand of a comma expression (7.6.20 [expr.comma]),
- the operand of a parenthesized expression (7.5.4 [expr.prim.paren]),
- a requirement in a requires expression (7.5.8.1 [expr.prim.req.general]),
- the second or third operand of `?:` (7.6.16 [expr.cond]),
- the operand of a typeid expression (7.6.1.8 [expr.typeid]),
- the operand of a noexcept operator (7.6.2.7 [expr.unary.noexcept]),
- the operand of a decltype specifier (9.2.9.6 [dcl.type.decltype]), or
- the operand of an explicit conversion to type `cv void` (7.6.1.4 [expr.type.conv], 7.6.1.9 [expr.static.cast], 7.6.3 [expr.cast]).

-- end note]

3105. Consteval destructor through immediate escalation

Section: 7.7 [[expr.const](#)] **Status:** ready **Submitter:** Corentin Jabot **Date:** 2025-10-14

(From submission [#780](#).)

Consider:

```
consteval void undefined();
template <typename T>
struct scope_exit {
    T t;
    constexpr ~scope_exit() { t(); }    // #2
};

scope_exit guard([]() {
    undefined();                        // #1
});
```

#1 is an immediate escalating expression (because `undefined` is not defined) and `~scope_exit()` is instantiated from a `constexpr` templated entity #2, thus is immediate escalating. Finally, this causes `~scope_exit` to be an immediate function.

However, destructors cannot be consteval.

Proposed resolution (approved by CWG 2025-11-04):

Change in 7.7 [[expr.const](#)] paragraph 26 as follows:

An immediate-escalating function is

- the call operator of a lambda that is not declared with the consteval specifier,
- a defaulted special member function that is not declared with the consteval specifier, or
- a function that **is not a prospective destructor and that** results from the instantiation of a templated entity defined with the `constexpr` specifier.

3106. Redundant exclusion of (non-existent) UCNs in *r-char-sequences*

Section: 5.3.2 [[lex.universal.char](#)] **Status:** ready **Submitter:** Hubert Tong **Date:** 2025-10-27

(From submission [#785](#).)

Subclause 5.3.2 [[lex.universal.char](#)] paragraph 1 specifically excludes *universal-character-names* within *r-char-sequences*, but *universal-character-names* are never formed there.

Proposed resolution (approved by CWG 2025-11-04):

Change in 5.3.2 [[lex.universal.char](#)] paragraph 1 as follows:

... If a *universal-character-name* outside the *c-char-sequence*, **or** *s-char-sequence*, ~~or *r-char-sequence*~~ of a *character-literal* or *string-literal* (in either case, including within a *user-defined-literal*) corresponds to a control character or to a character in the basic character set, the program is ill-formed. ...

3107. Misleading note "An alias template name is never deduced."

Section: 13.7.8 [[temp.alias](#)] **Status:** ready **Submitter:** Brian Bi **Date:** 2025-11-02

(From submission [#786](#).)

The note in 13.7.8 [temp.alias] paragraph 2 is incorrect because an alias template A can indeed be deduced from a type of the form T<A> (where T is a template with a type template template parameter).

Proposed resolution (approved by CWG 2025-11-04):

Change in 13.7.8 [temp.alias] paragraph 2 as follows:

... [Note 1: An **The** alias template name is **never not** deduced **from such a type (13.10.3.6 [temp.deduct.type])**. —end note] ...

3108. Reflection on type aliases

Section: 7.6.2.10 [expr.reflect] **Status:** ready **Submitter:** Jakub Jelinek **Date:** 2025-10-29

(From submission #789.)

It is not clear in 7.6.2.10 [expr.reflect] paragraph 6 whether the phrasing "names a type alias" also applies for type aliases appearing somewhere in a *type-id*, e.g. `constexpr my_typedef_name`.

Proposed resolution (approved by CWG 2025-11-04):

Change in 7.6.2.10 [expr.reflect] bullet 6.2 as follows:

A *reflect-expression* R of the form `type-id` represents an entity determined as follows:

- If the *type-id* designates a placeholder type (9.2.9.7.1 [dcl.spec.auto.general]), R is ill-formed.
- Otherwise, if the *type-id* **names a type alias that is a specialization of is of the form *nested-name-specifier*_{opt} *template*_{opt} *simple-template-id* and whose terminal name is a *template-name* that names** an alias template (13.7.8 [temp.alias]), R represents **that the** type alias **so named**.
- Otherwise, R represents the type denoted by the *type-id*.

3109. Access checking when designating a protected member by a splice

Section: 11.8.5 [class.protected] **Status:** ready **Submitter:** Dan Katz **Date:** 2025-10-25

(From submission #784.)

Subclause 11.8.5 [class.protected] establishes an "additional rule" for access to protected non-static members. That rule should be disabled for members designated by a *splice-expression*.

Proposed resolution (approved by CWG 2025-11-04):

Change in 11.8.5 [class.protected] as follows:

An additional access check beyond those described earlier in 11.8 [class.access] is applied when a non-static data member or non-static member function is a protected member of its designating class (11.8.3 [class.access.base]) **and is not designated by a *splice-expression***. [Footnote: ...] As described earlier, access to a protected member is granted because the reference occurs in a friend or direct member of some class C. If the access is to form a pointer to member (7.6.2.2 [expr.unary.op]), the *nested-name-specifier* shall **denote designate** C or a class derived from C. **All other accesses involve** **Otherwise, if the access involves** a (possibly implicit) object expression (7.5.5.1 [expr.prim.id.general], 7.6.1.5 [expr.ref]). **In this case**, the class of the object expression shall be C or a class derived from C.

3110. Constexpr allocation for literal types

Section: 7.7 [expr.const] **Status:** ready **Submitter:** Jiang An **Date:** 2025-11-01

(From submission #792.)

Consider:

```
#include <locale>
#include <memory>

static_assert([]{
    auto a = std::allocator<std::locale>{};
    a.deallocate(a.allocate(42), 42);
    return true;
})();
```

The type `std::locale` is not a literal type, yet this is accepted by all implementations.

Proposed resolution (approved by CWG 2025-11-04):

Change in 7.7 [expr.const] paragraph 14 as follows:

For the purposes of determining whether an expression E is a core constant expression, the evaluation of the body of a member function of `std::allocator<T>` as defined in 20.2.10.2 [allocator.members], ~~where T is a literal type~~, is ignored.

3111. Template parameter objects of array type

Section: 13.2 [temp.param] **Status:** ready **Submitter:** Dan Katz **Date:** 2025-10-30

(From submission #791.)

Constant template parameters of array type decay to pointers, thus template parameter objects of array type are never created this way. However, with reflection, 21.4.3 [meta.define.static] paragraph 11 does ask for such objects, which are underspecified.

Proposed resolution (approved by CWG 2025-11-07):

1. Change in 13.2 [temp.param] paragraph 13 as follows:

Certain constructs refer to *template parameter objects*, which are distinct objects with static storage duration and non-volatile const type. No two such objects have template-argument-equivalent values (13.6 [temp.type]). An *id-expression* naming a constant template parameter of class type T denotes ~~a static storage duration~~ **the template parameter** object of type `const T`, ~~known as a template parameter object~~, which is template-argument-equivalent (13.6 [temp.type]) to the corresponding template argument after it has been converted to the type of the template parameter (13.4.3 [temp.arg.nontype]). ~~No two template parameter objects are template-argument-equivalent.~~

[Note: There can be template parameter objects of array type (21.4.3 [meta.define.static]), but such an object is never denoted by an *id-expression* that names a constant template parameter. -- end note]

[Note 2: If an *id-expression* names a non-reference constant template parameter, then it is a prvalue if it has non-class type. Otherwise, if it is of class type T, it is an lvalue and has type `const T` (7.5.5.2 [expr.prim.id.unqual]). —end note] [Example: ...]

2. Change in 21.4.7 [meta.reflection.queries] paragraph 7 as follows:

Effects: Equivalent to:

```
if constexpr (is_annotation(R)) {
    return C;
} else if constexpr (is_array_type(type_of(R))) {
    return reflect_constant_array([: R :]);
} else if constexpr (is_function_type(type_of(R))) {
    return reflect_function([: R :]);
} else {
    return reflect_constant([: R :]);
}
```

3112. Introduce a term for C-style variadic functions

Section: 13.2 [temp.param] **Status:** ready **Submitter:** FR **Date:** 2025-10-01 [N5028 comment FR 155](#)

Introduce a term for C-style variadic functions, i.e. those where the parameter-type-list ends with an ellipsis.

Proposed resolution (approved by CWG 2025-11-04):

Insert a paragraph before 9.3.4.6 [dcl.fct] paragraph 4 as follows:

A function with a parameter-type-list that has an ellipsis is termed a *vararg function*.

An *explicit-object-parameter-declaration* is a *parameter-declaration* with a *this* specifier. ...

3113. When is an expansion-init-list type-dependent?

Section: 13.8.3.3 [temp.dep.expr] **Status:** ready **Submitter:** US **Date:** 2025-10-01 [N5028 comment US 097](#)

Subclause 13.8.3.3 [temp.dep.expr] bullet 3.11 gives the impression that the specification for a type-dependent *expansion-init-list* is missing.

Proposed resolution (approved by CWG 2025-11-04):

Change in 13.8.3.3 [temp.dep.expr] bullet 3.11 as follows:

- ...
- a name N introduced by the *for-range-declaration* of an *expansion-statement* S if the type specified for N contains a placeholder type and either
 - S is not an iterating expansion statement or
 - the *expansion-initializer* of S is type-dependent, or
 - ~~S is not an iterating expansion statement, or~~
- ...

3114. Indirect base classes for class member access with direct base class relationship

Section: 7.6.1.5 [expr.ref] **Status:** ready **Submitter:** CWG **Date:** 2025-11-03

Subclause 7.6.1.5 [expr.ref] bullet 8.6 does not specify what happens if the direct base class relationship originates from an unambiguous base class of E1.

Proposed resolution (approved by CWG 2025-11-07):

Change in 7.6.1.5 [expr.ref] bullet 8.6 as follows:

- Otherwise, if E2 designates a direct base class relationship (*D*, *B*) and *D* is either the **cv-unqualified class** type of E1 ~~is *cv T* or a base class thereof~~, let *cv* be the cv-qualification of the type of E1. E1 is implicitly converted to the type "reference to *cv D*" (where the reference is an lvalue reference if E1 is an lvalue and an rvalue reference otherwise) and the expression designates the direct base class subobject of type B of the object designated by the converted E1. If E1 is an lvalue, then E1.E2 is an lvalue; otherwise, E1.E2 is an xvalue. The type of E1.E2 is "*cv B*". ~~[Note 6: This can only occur in an expression of the form *e1*. [*:e2*];. — end note]~~

3115. Function parameters of consteval-only type

Section: 6.9.1 [basic.types.general] **Status:** ready **Submitter:** Hubert Tong **Date:** 2025-10-29

The properties of functions with consteval-only parameters are unclear.

Proposed resolution (approved by CWG 2025-11-04):

1. Change in 6.9.1 [basic.types.general] paragraph 12 as follows:

A type is consteval-only if it is either `std::meta::info` or a type compounded from a consteval-only type (6.9.4 [basic.compound]). Every object of consteval-only type shall be

- the object associated with a `constexpr` variable or a subobject thereof,
- a template parameter object (13.2 [temp.param]) or a subobject thereof, or
- an object whose lifetime begins and ends during the evaluation of a core constant expression.

Every function of consteval-only type shall be an immediate function (7.7 [expr.const]).

2. Change in 7.7 [expr.const] paragraph 27 as follows:

An immediate function is a function that is ~~either~~

- declared with the consteval specifier, ~~or~~
- an immediate-escalating function whose type is consteval-only (6.9.1 [basic.types.general]), or
- an immediate-escalating function *F* whose function body contains either
 - an immediate-escalating expression or
 - a definition of a non-`constexpr` variable with consteval-only type

whose innermost enclosing non-block scope is *F*'s function parameter scope. [Note 11: Default member initializers used to initialize a base or member subobject (11.9.3 [class.base.init]) are considered to be part of the function body (9.6.1 [decl.fct.def.general]). —end note]

...

3116. First element of an array

Section: 9.3.4.5 [decl.array] **Status:** ready **Submitter:** CWG **Date:** 2025-11-05

It is unclear whether the phrase "first element" of an array refers to the initial element or to the element with subscript 1.

Proposed resolution (approved by CWG 2025-11-06):

Change in 9.3.4.5 [decl.array] paragraph 6 as follows:

An object of type "array of *N* *U*" consists of a contiguously allocated non-empty set of *N* subobjects of type *U*, known as the elements of the array, and numbered 0 to *N*-1. **The element numbered 0 is termed the first element of the array.**

3117. Overriding by a consteval virtual function

Section: 11.7.3 [class.virtual] **Status:** ready **Submitter:** Daniel Katz **Date:** 2025-11-06

The current prohibition against overriding a non-consteval virtual function by a consteval virtual function (inside a consteval-only type) is too restrictive and prevents implementation of `std::meta::exception`.

Proposed resolution (approved by CWG 2025-11-06):

Change in 11.7.3 [class.virtual] paragraph 18 as follows:

A class with a consteval virtual function ~~shall not override~~ that overrides a virtual function that is not consteval **shall have consteval-only type (6.9.1 [basic.types.general])**. A consteval virtual function shall not be overridden by a virtual function that is not consteval.

3118. Mangling reflections of annotations is infeasible

Section: 6.7 [basic.link] **Status:** ready **Submitter:** Dan Katz **Date:** 2025-11-06

(From submission #796.)

Consider:

```
[[=1]] void f();  
constexpr auto R = annotations_of(^^f)[0];  
  
template <std::meta::info> struct TCls {};
```

R could be exported across module boundaries, which would require TCls<R> to be mangled.

Proposed resolution (approved by CWG 2025-11-07):

Change in 6.7 [[basic.link](#)] bullet 16.4 as follows:

- ...
- it is a reflection value (6.9.2 [[basic.fundamental](#)]) that represents
 - an entity, value, or object that is TU-local,
 - an annotation (9.13.12 [[dcl.attr.annotation](#)]),
 - a direct base class relationship (D, B) (11.7.1 [[class.derived.general](#)]) for which either D or B is TU-local, or
 - a data member description (T, N, A, W, NUA) (11.4.1 [[class.mem.general](#)]) for which T is TU-local.